Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

Introduction

This report documents the steps I followed to complete the cloud database and API assignment using a PostgreSQL database hosted on Neon, DbGate Community, and a FastAPI application. The main goal of the assignment was to practice working with a relational database in a cloud environment and to connect it to a backend API that performs real database operations. The work focused on setting up the database, writing SQL queries, and building API endpoints that interact with the database correctly. Screenshots are included throughout the report to show the results and confirm that each task was completed successfully.

The assignment was divided into several steps. I started by creating a cloud PostgreSQL database and connecting to it using DbGate. After that, I created the database schema and inserted sample data using SQL files. Once the database was ready, I ran the FastAPI application locally and verified that it could connect to the database. I then implemented API endpoints to retrieve products using SQL joins, create orders with stock updates, and generate summary statistics using aggregation and grouping queries.

During the assignment, I faced a few challenges. One issue was making sure that database connections were correctly configured inside the Docker environment. Another challenge was handling transactions when creating orders, so that stock levels, orders, and order items were updated safely without leaving partial data. I also needed to carefully test SQL joins and aggregate queries to ensure that the API responses matched the results shown in DbGate.

Step 1 — Create Neon PostgreSQL + connect with DbGate

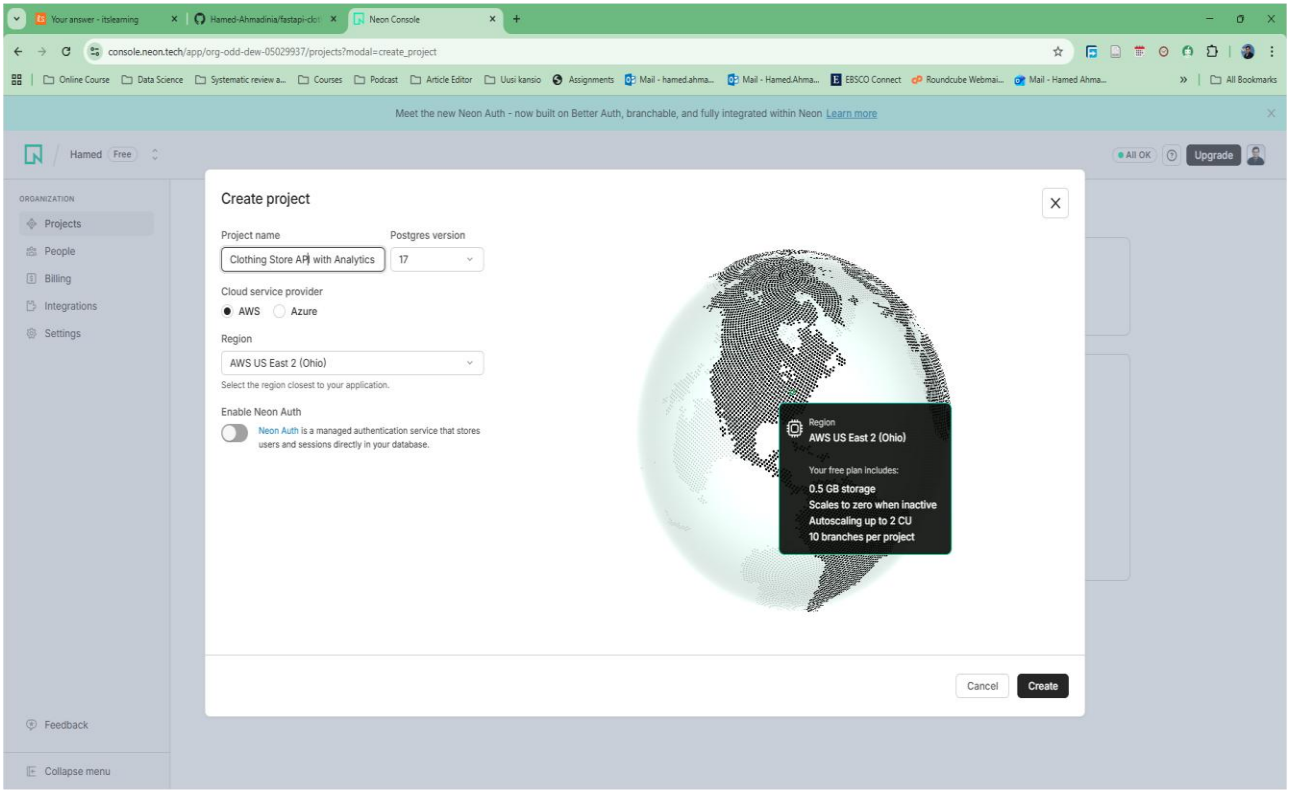
In the first step, I set up the database environment that was required for all later parts of the assignment. I started by creating a new PostgreSQL project using the Neon cloud database platform. During the setup, I selected PostgreSQL version 17 and chose the AWS US East (Ohio) region. After the project was created, I obtained the database connection string and credentials provided by Neon.

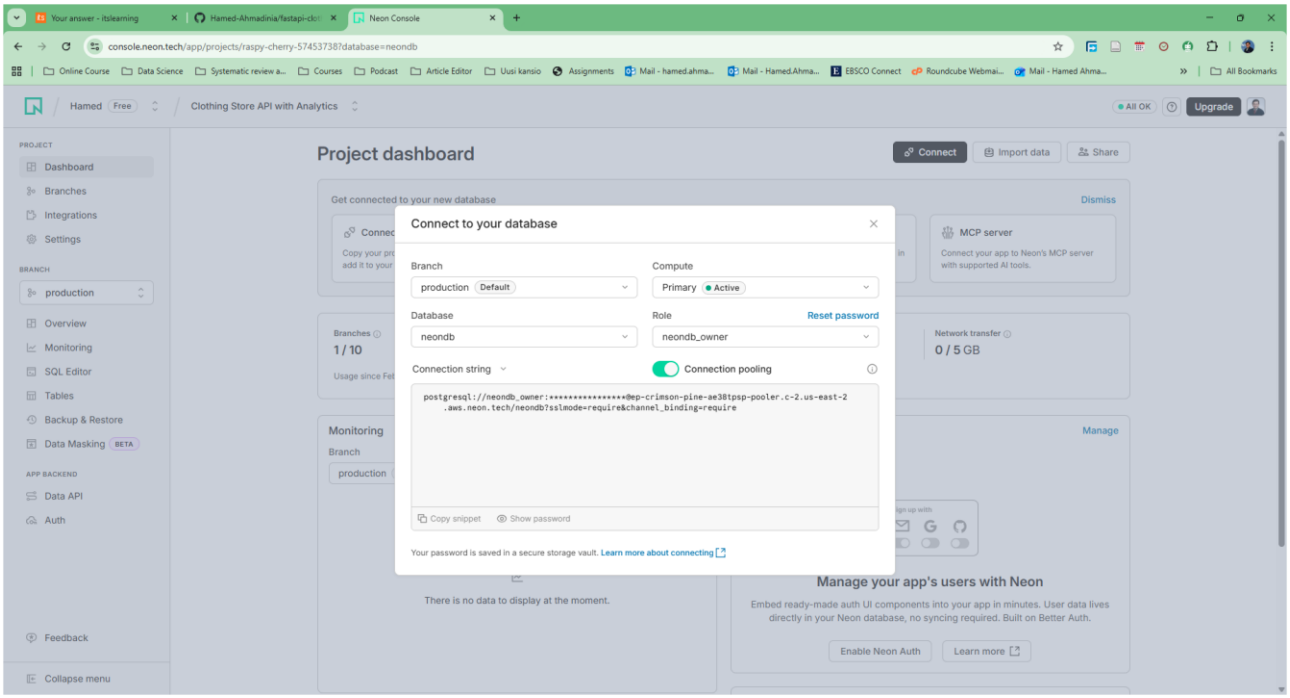
Next, I opened DbGate Community and created a new database connection. In the connection settings, I selected PostgreSQL as the database type and used the Neon connection string, username, and password. I tested the connection to ensure it was successful and then saved it.

Once connected, I verified the setup by running a simple SQL query to check the current database and user. I also confirmed that the database tables (such as categories, products, customers, orders, and order_items) were visible in the left panel. This confirmed that the database connection was working correctly and ready for executing SQL queries in the following steps. At this stage, the database connection was confirmed, and the environment was ready for schema creation in the next step.

Assignment 5: Clothing Store API with Analytics

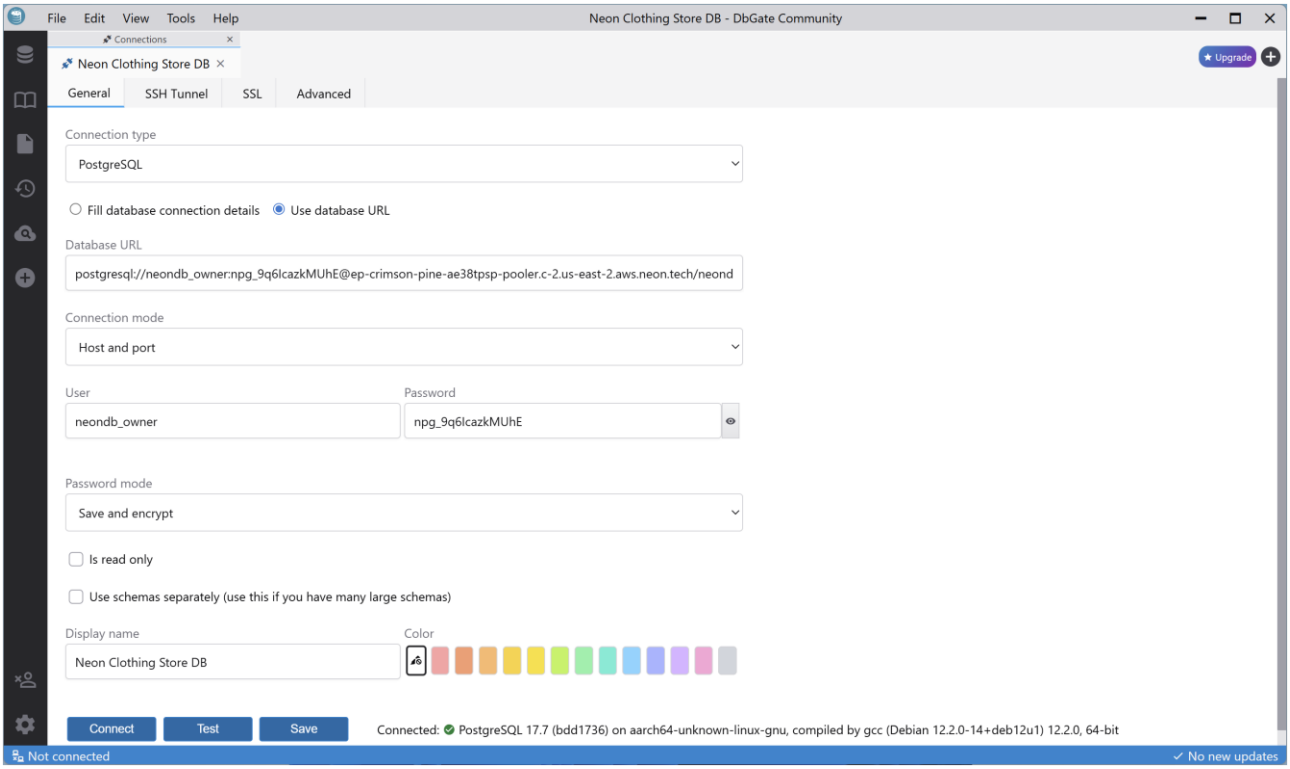
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

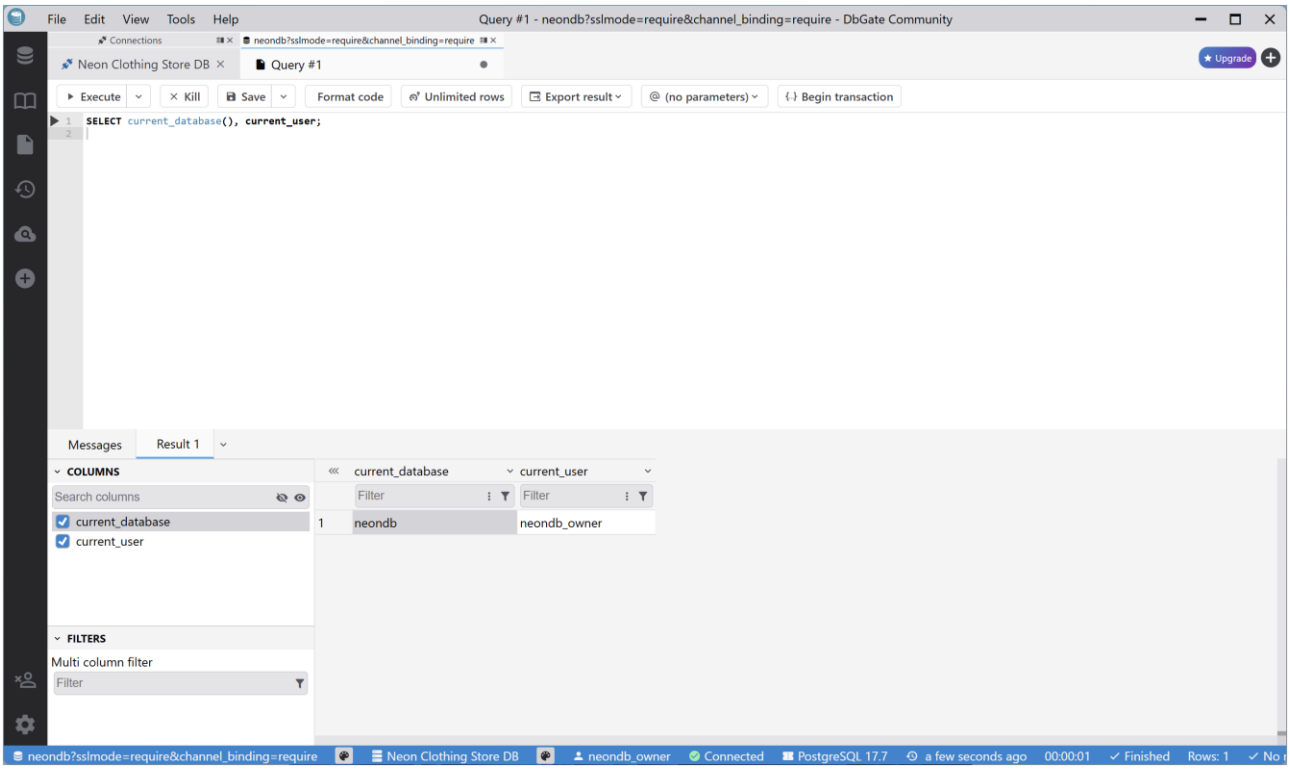




Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





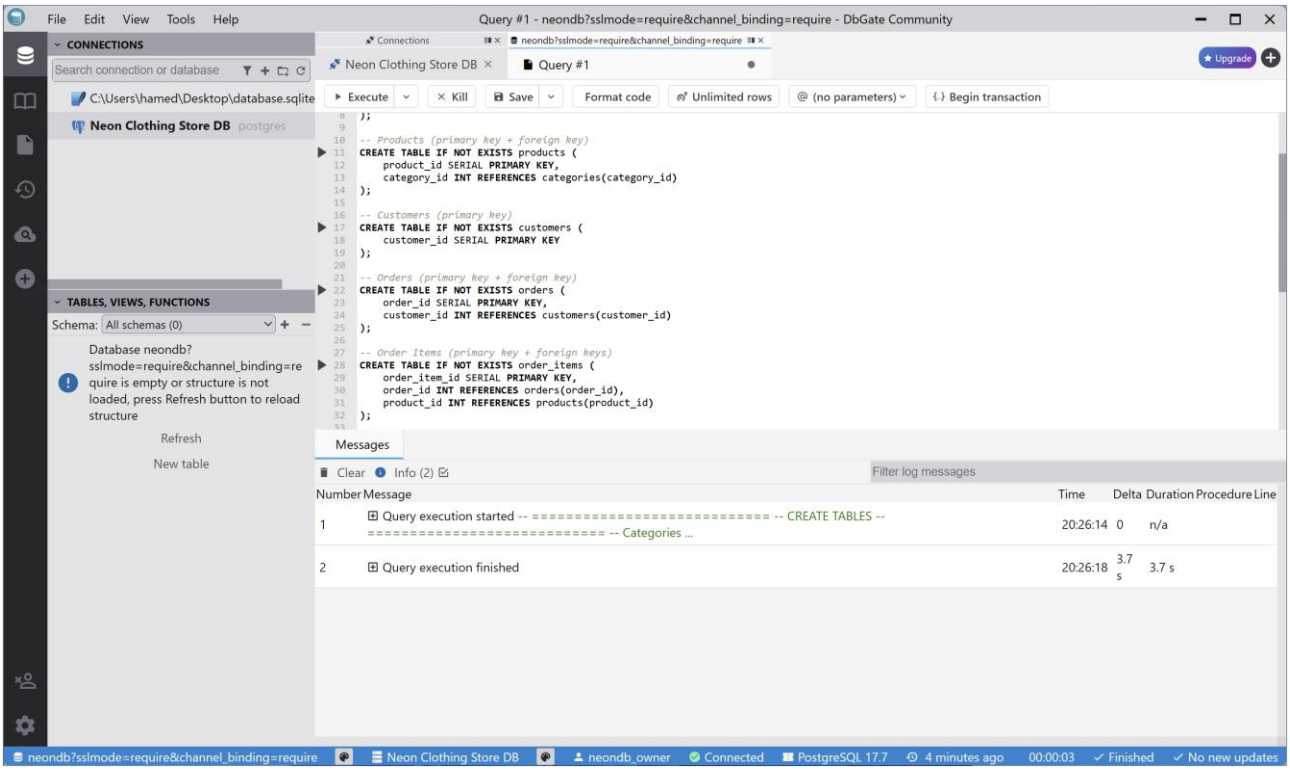
Step 2 — Seed schema + sample data (DbGate)

In the second step, I used DbGate Community to create the database schema and insert sample data needed for the application. After connecting to the Neon PostgreSQL database, I opened a new SQL query window in DbGate.

First, I opened the provided SQL files from the template repository and executed them in DbGate by copying and running them in order. Each table was created with a primary key, and foreign key relationships were added to link related tables correctly, such as products to categories and orders to customers. After running the queries, I confirmed that all tables were created successfully by checking that they appeared in the tables list on the left panel.

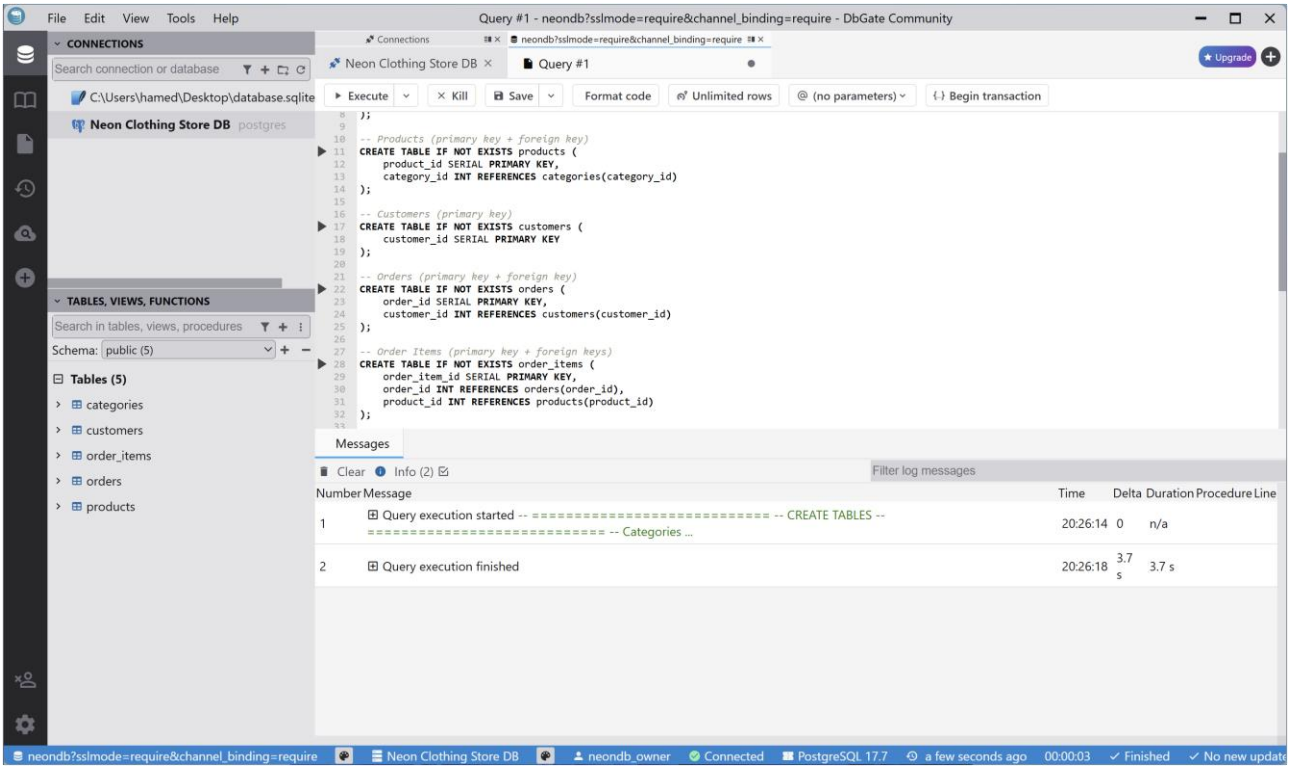
Next, I inserted sample data into the tables using INSERT statements. This included example categories, products, customers, orders, and order items. DbGate confirmed that the rows were inserted without errors and displayed the number of affected rows for each insert operation.

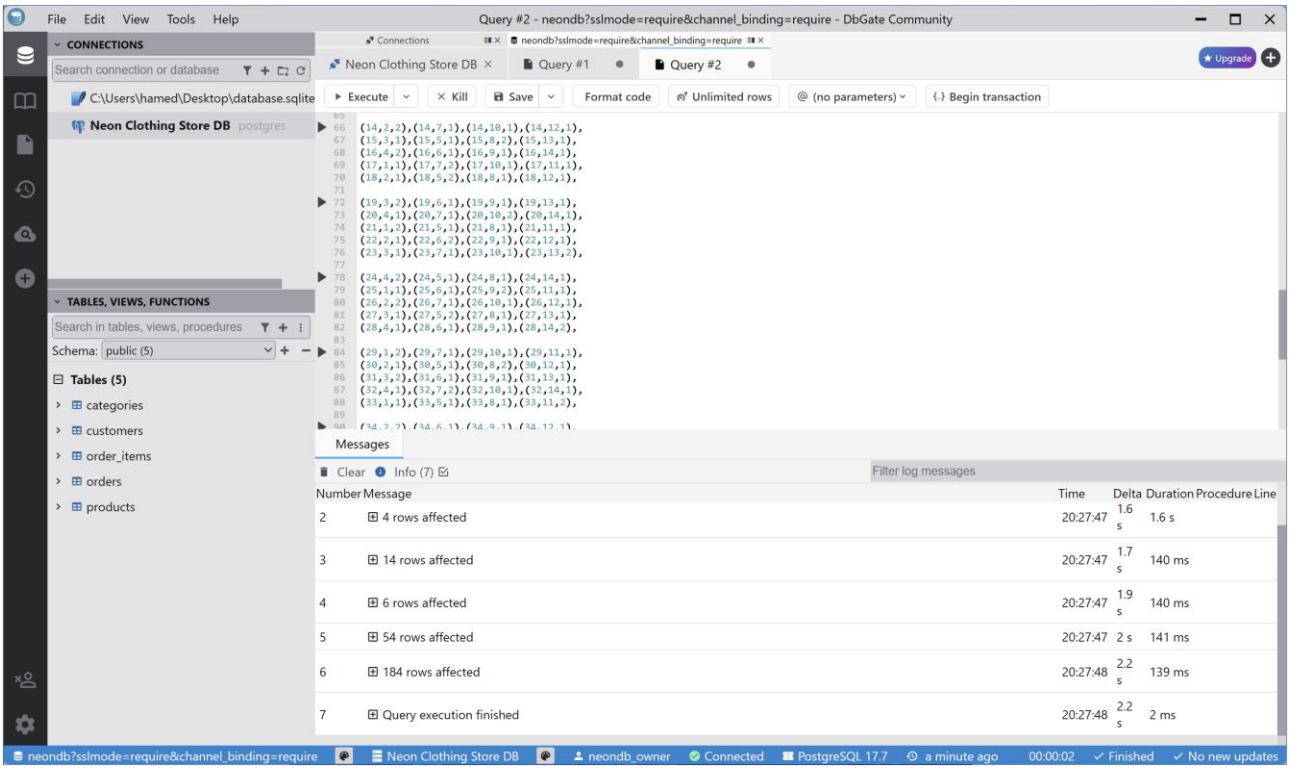
Finally, I verified the data by running a summary query that counted the number of rows in each table. The results showed that all tables contained data, confirming that the schema and sample data were correctly set up. This completed the database seeding step and prepared the database for use by the FastAPI application in later steps.



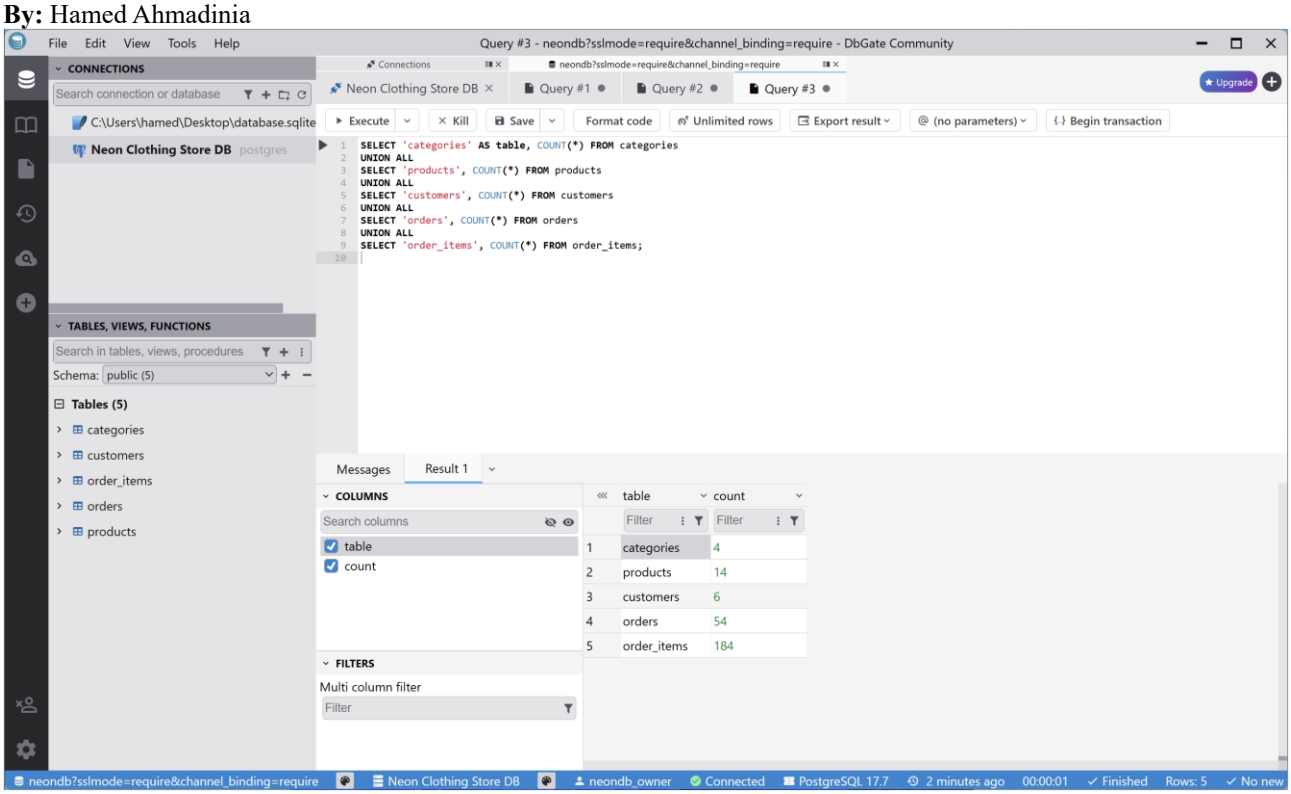
Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia



Step 3 — Local FastAPI runs + DB connectivity verified

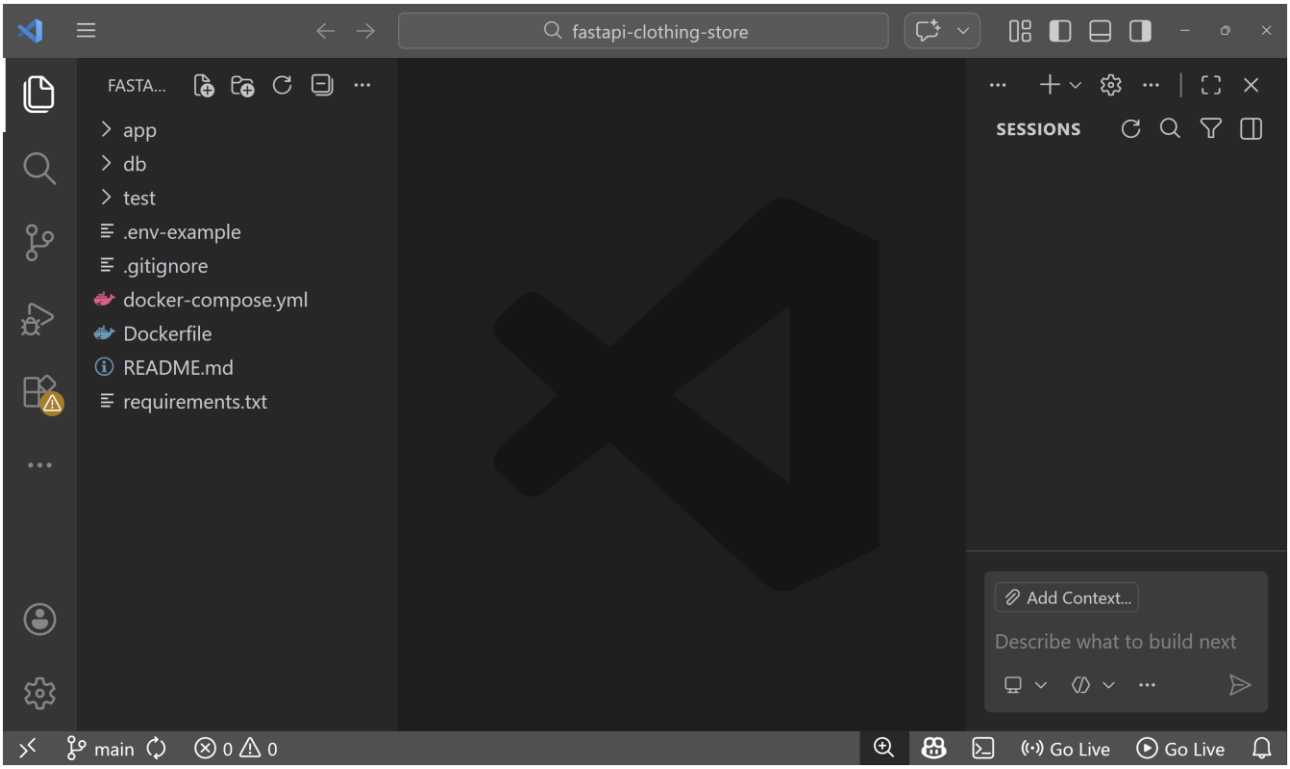
In this step, I configured and verified the local FastAPI application and confirmed that it could successfully connect to the PostgreSQL database. I first created a .env file in the project root and added the required environment variables, including the application mode and the database connection URL provided by Neon.

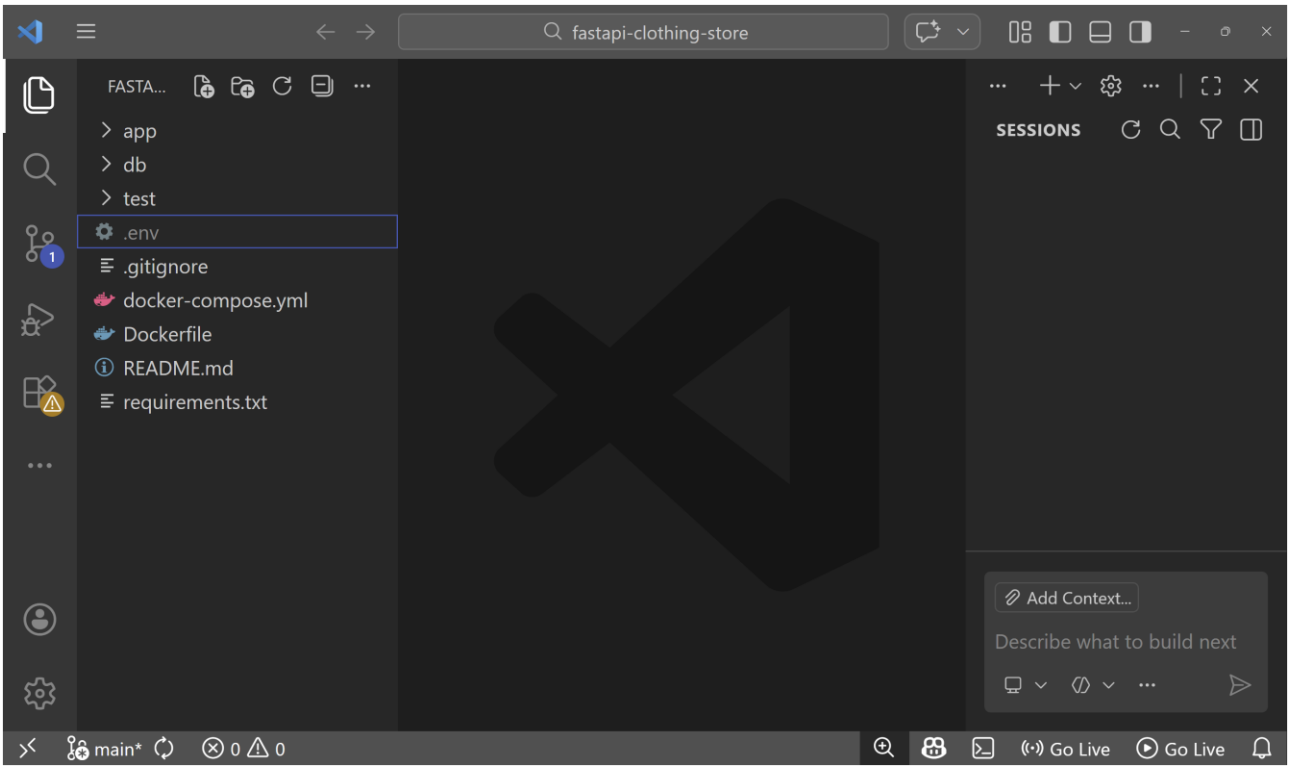
After setting the environment variables, I started the FastAPI application using Docker Compose. The build process completed successfully, and the container started without errors. The terminal logs confirmed that Uvicorn was running and listening on port 8080, and the application startup completed normally.

Once the container was running, I opened the FastAPI application in the browser and verified that the root endpoint responded correctly. I also accessed the Swagger UI to confirm that the API endpoints were available and loaded without issues.

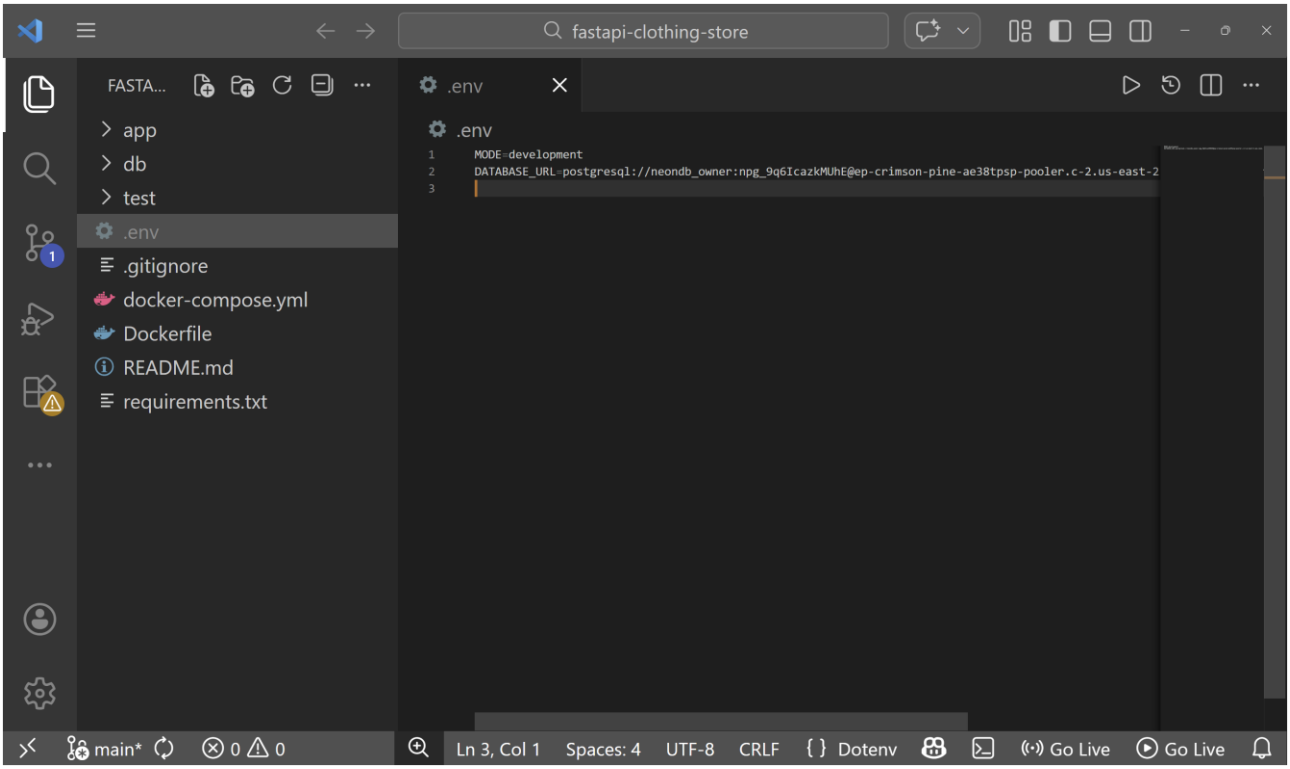
To ensure database connectivity from the application, I executed a database check from inside the running FastAPI container. The query returned the correct database name and user, confirming that the application was connected to the Neon PostgreSQL database as expected. This verified that the local FastAPI setup and database connection were working correctly and ready for further development and testing.

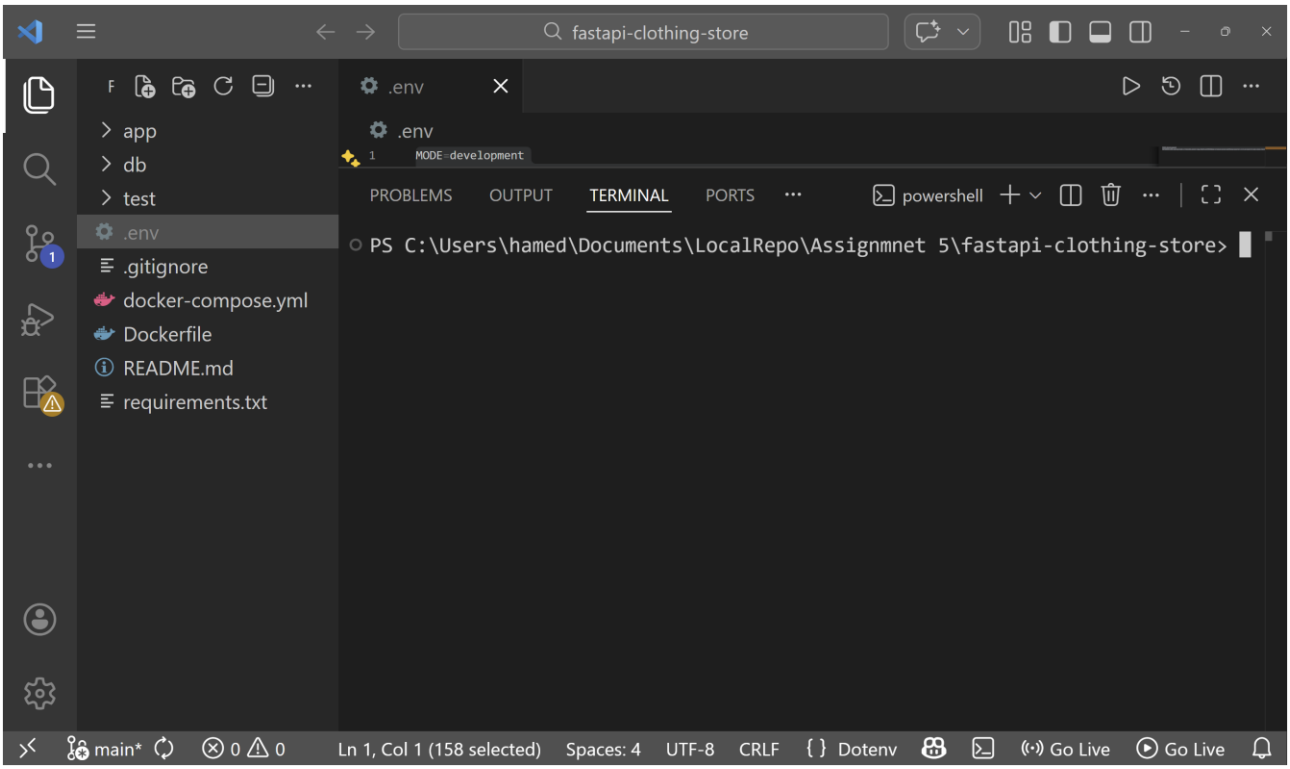
Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia



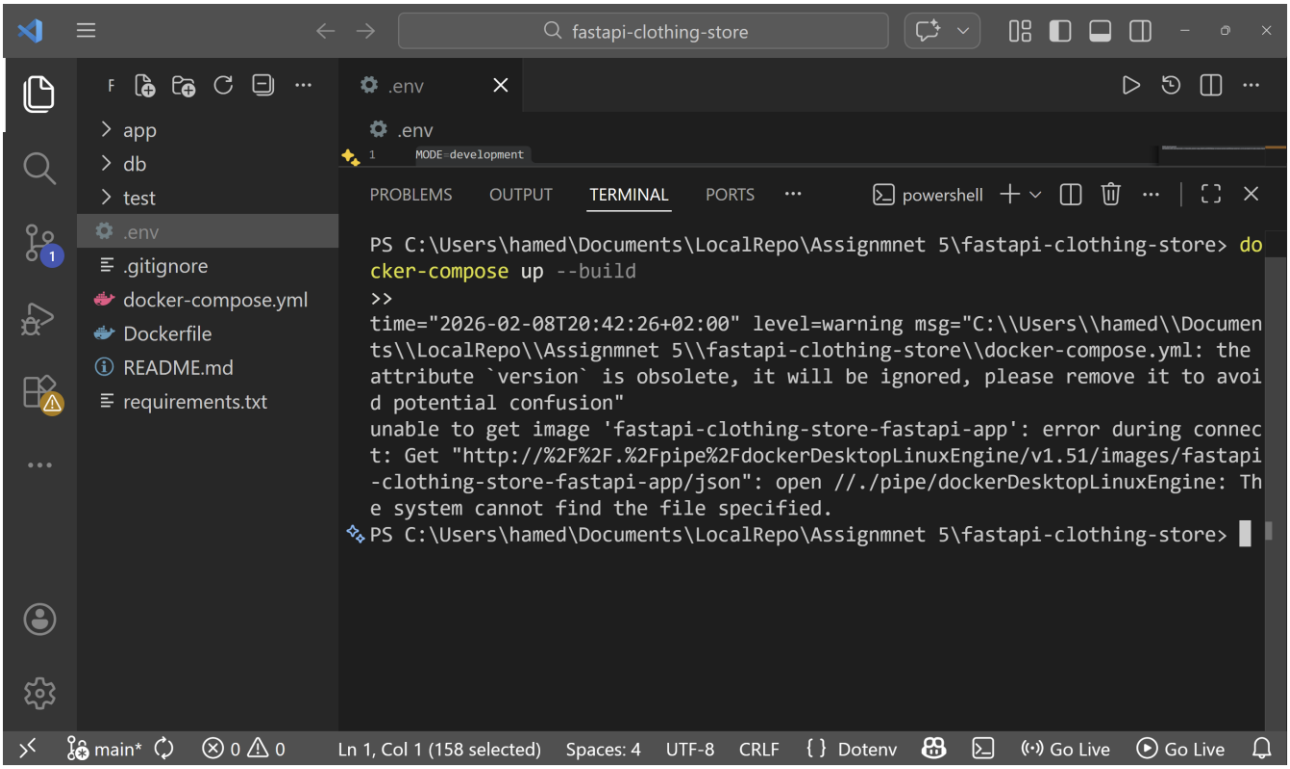


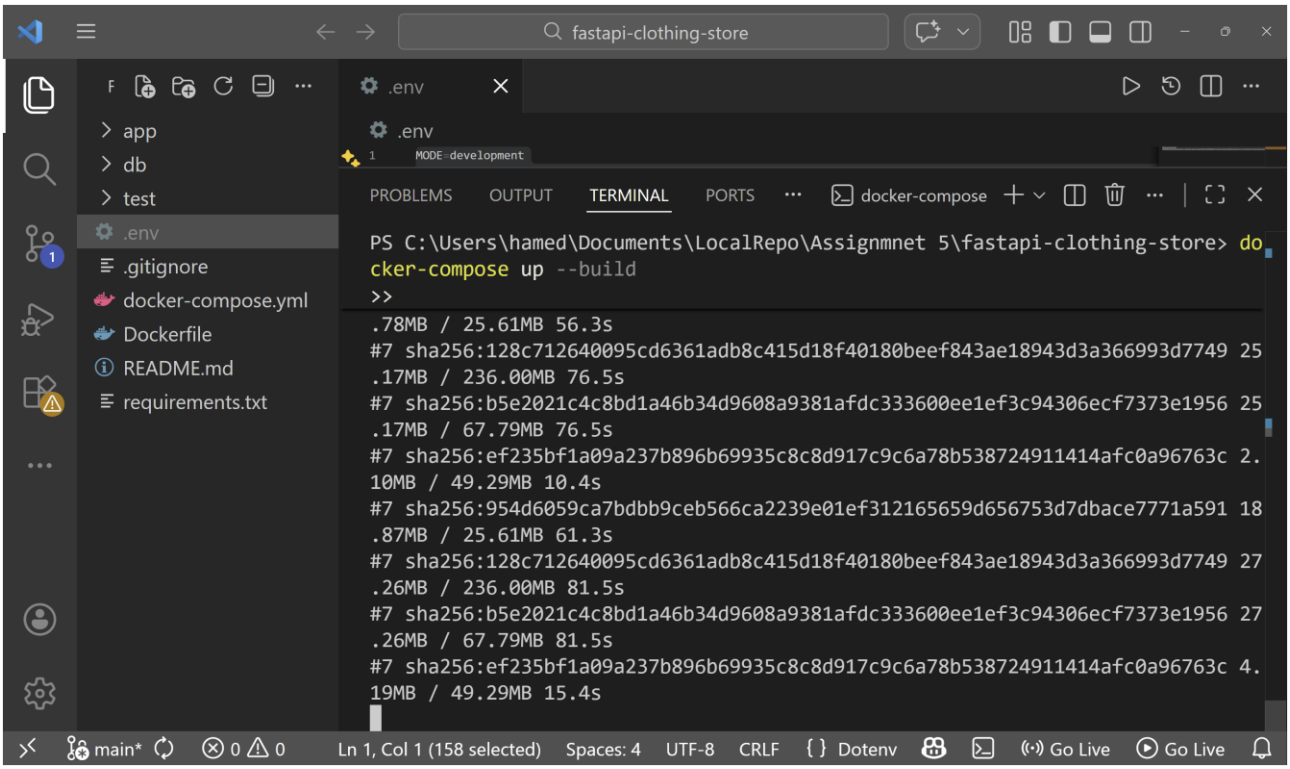
Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





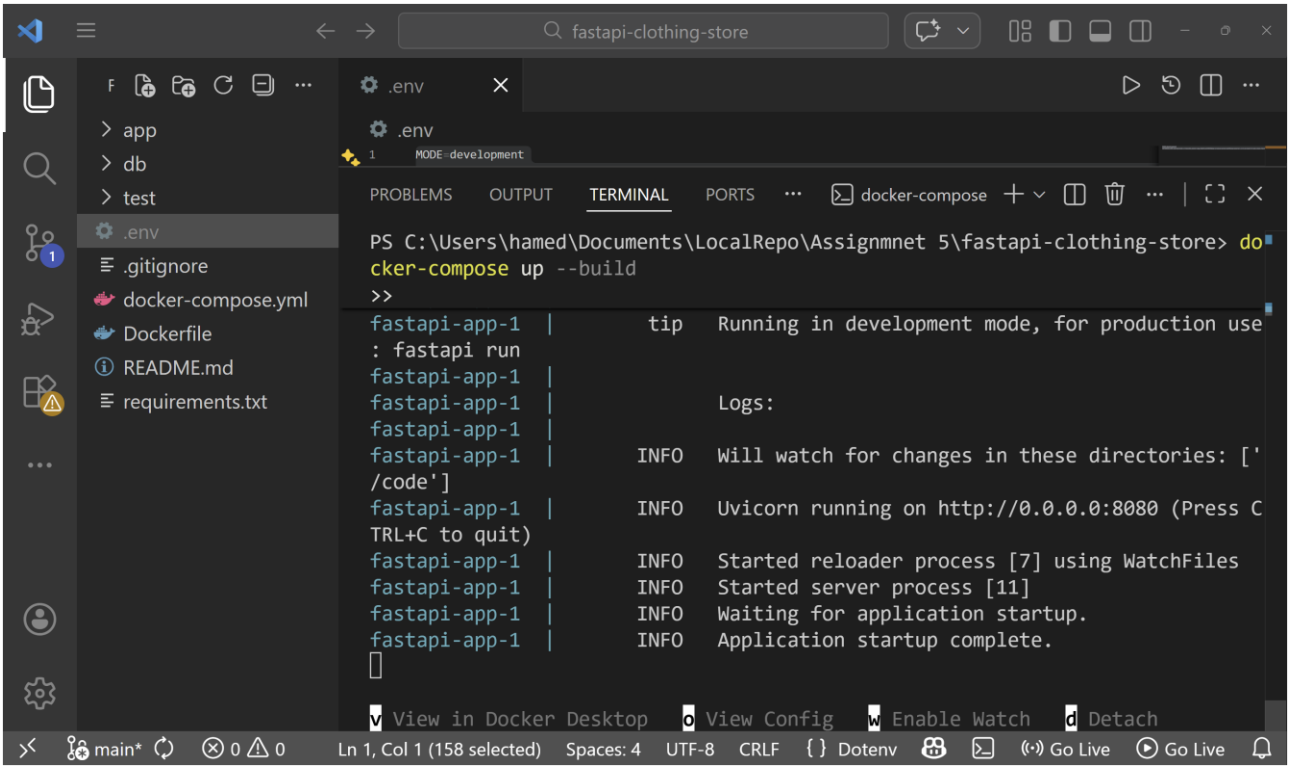
Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

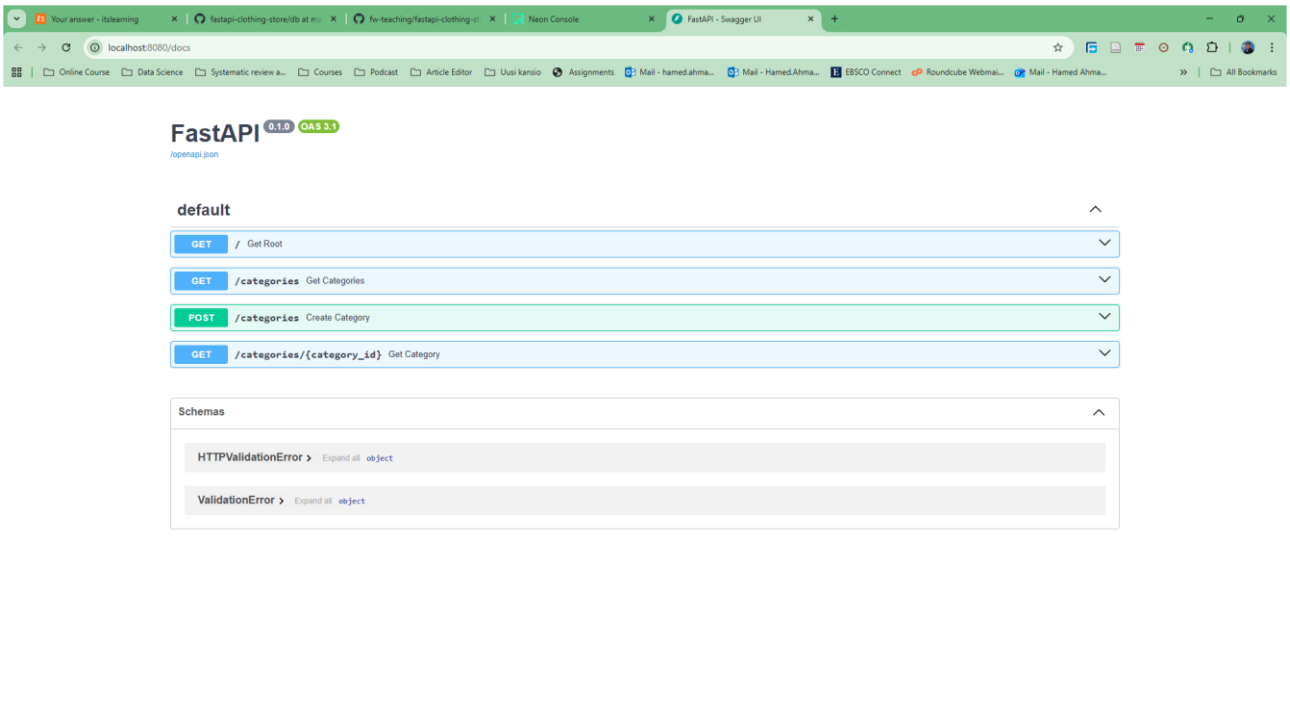




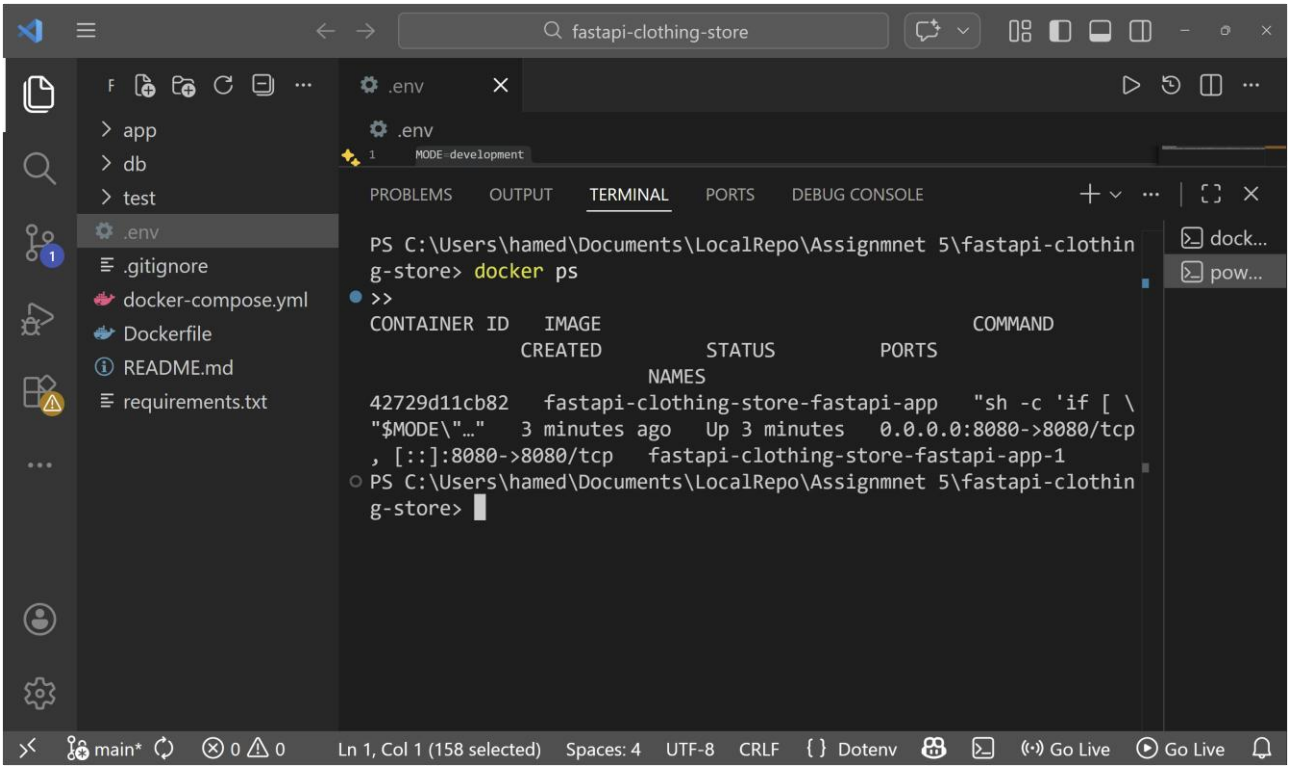
Assignment 5: Clothing Store API with Analytics

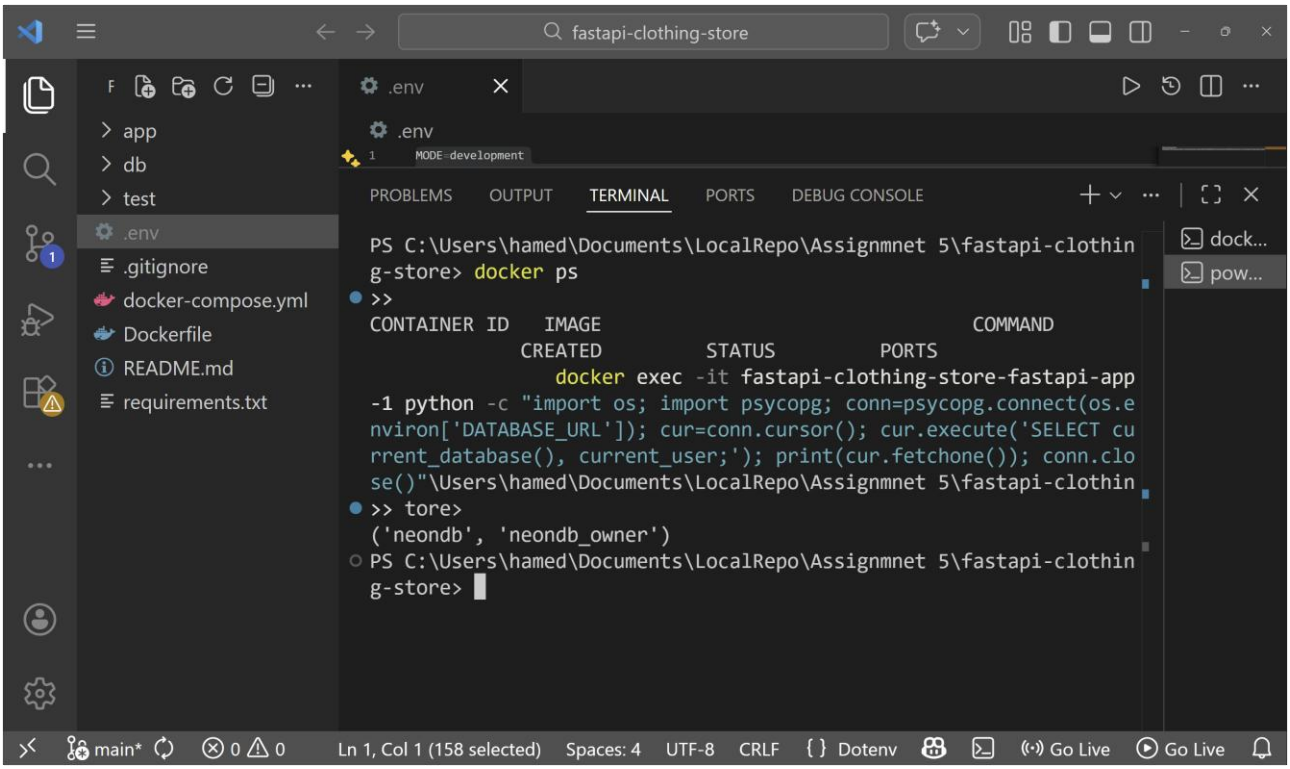
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia



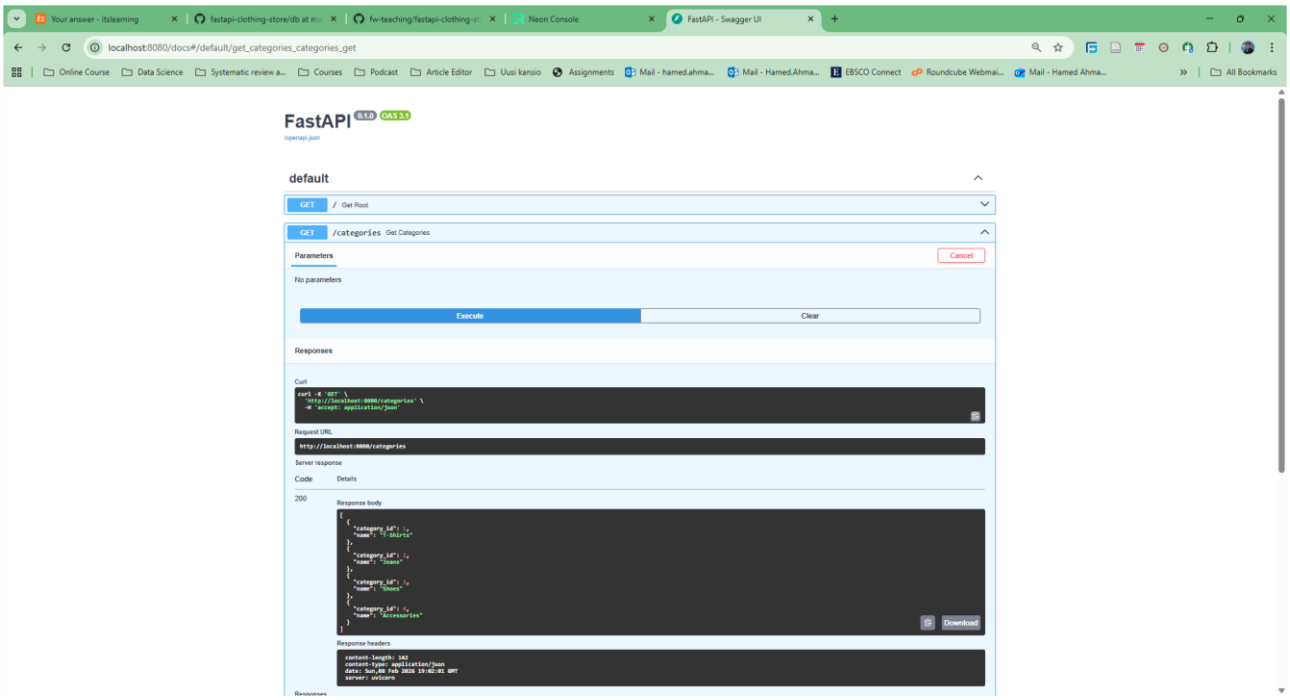


Step 4 — Implement GET /products (JOIN)

In this step, I implemented the `GET /products` endpoint to retrieve product data together with its related category information. Since products and categories are stored in separate tables, I used a SQL `JOIN` to combine data from both tables in a single query.

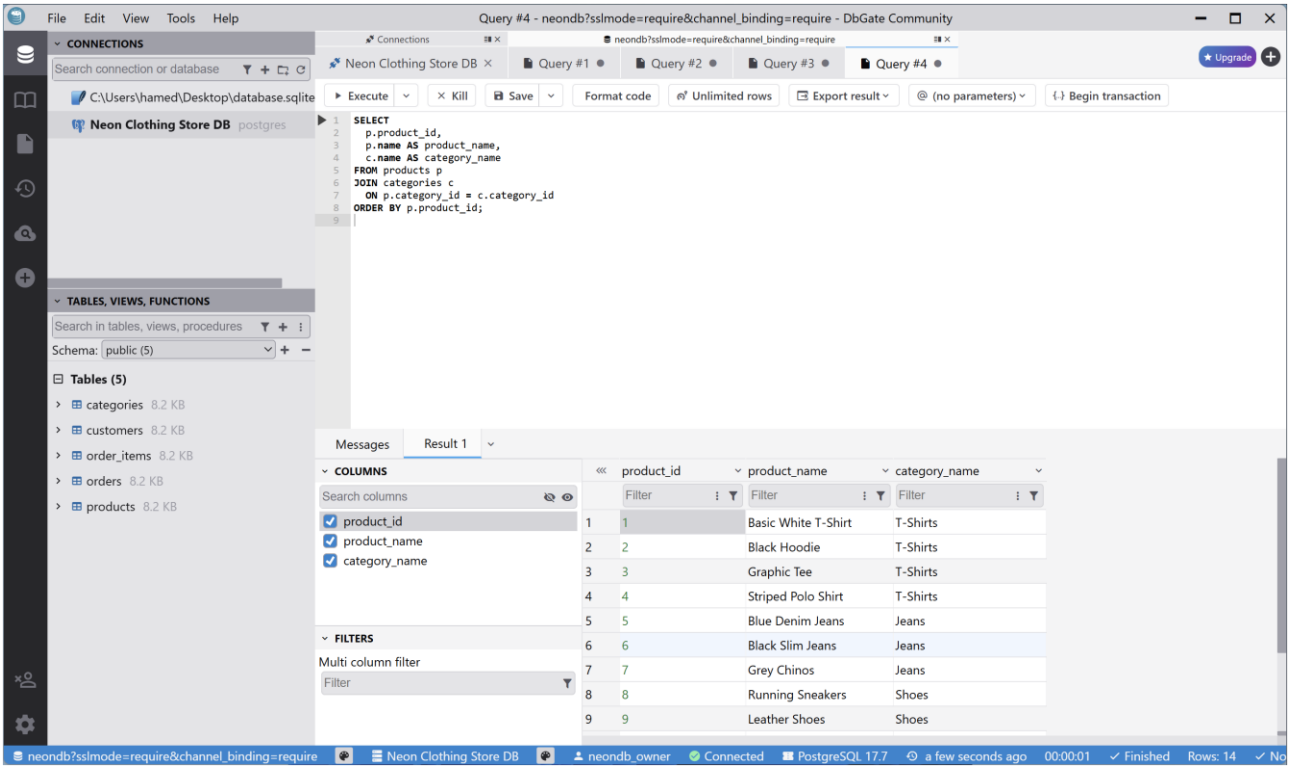
The query joins the `products` table with the `categories` table using the shared `category_id` field. This allows each product to be returned along with its category name instead of only the category ID. The query was executed through the database connection used by the FastAPI application. The response includes product name, category name, price, and stock level.

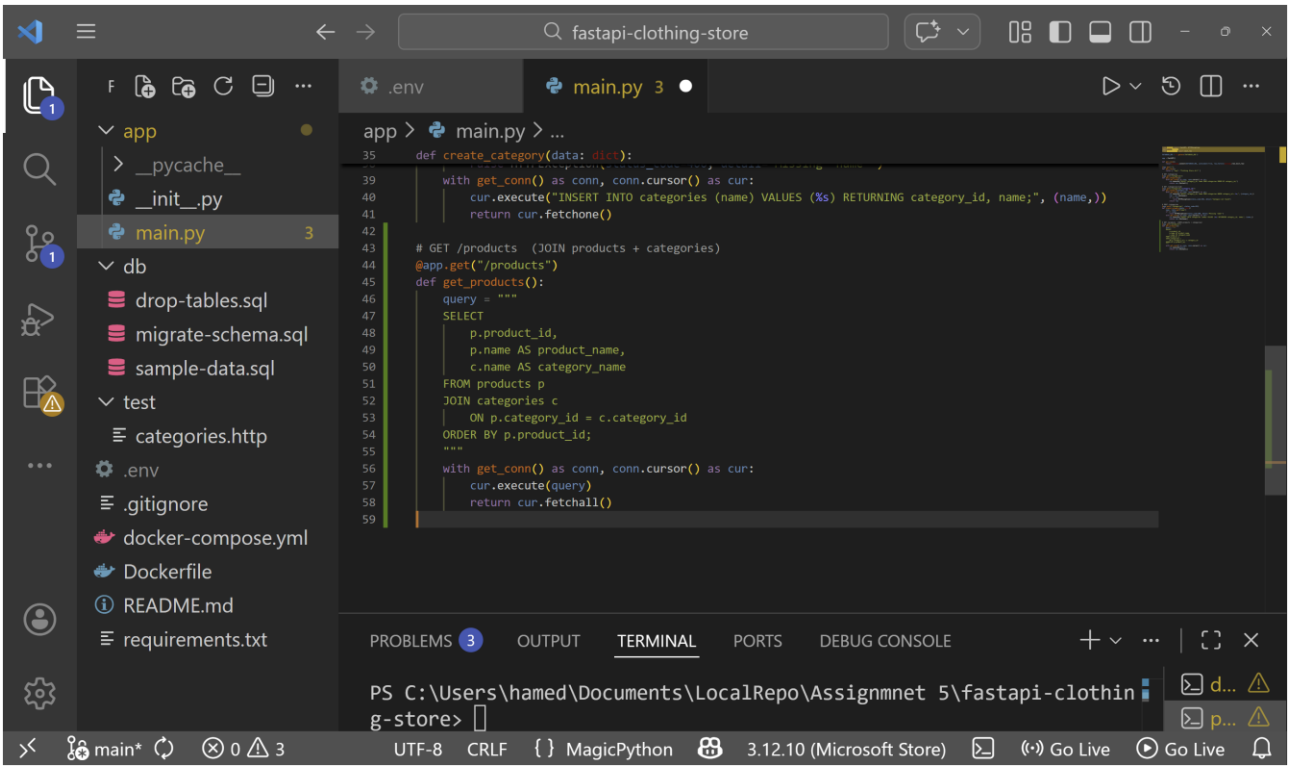
After implementing the endpoint, I tested it using the Swagger UI. The endpoint returned a list of products with their associated category details, confirming that the join operation worked correctly. This step demonstrated the use of relational database concepts within the API and ensured that product data is returned in a more meaningful and user-friendly format.

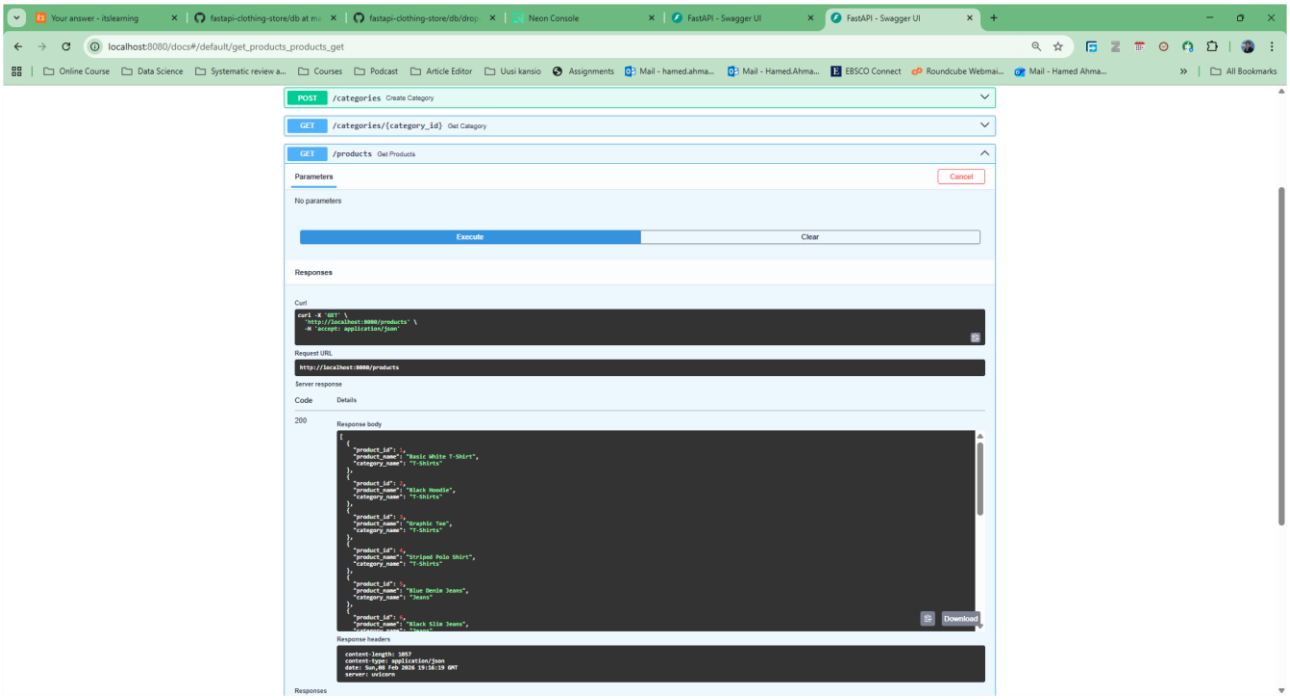


Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia







Step 5 — Implement POST /orders with transaction + stock decrement

In this step, I created the POST /orders endpoint to place an order and update stock in the database. The goal was to make sure an order is only saved if every item in the request is valid and there is enough stock for each product.

First, the endpoint reads customer_id and the items list from the request body. I added input checks so the request is rejected if the customer ID is missing or the items list is empty. Then, the code opens a database connection and starts a manual transaction by turning off autocommit.

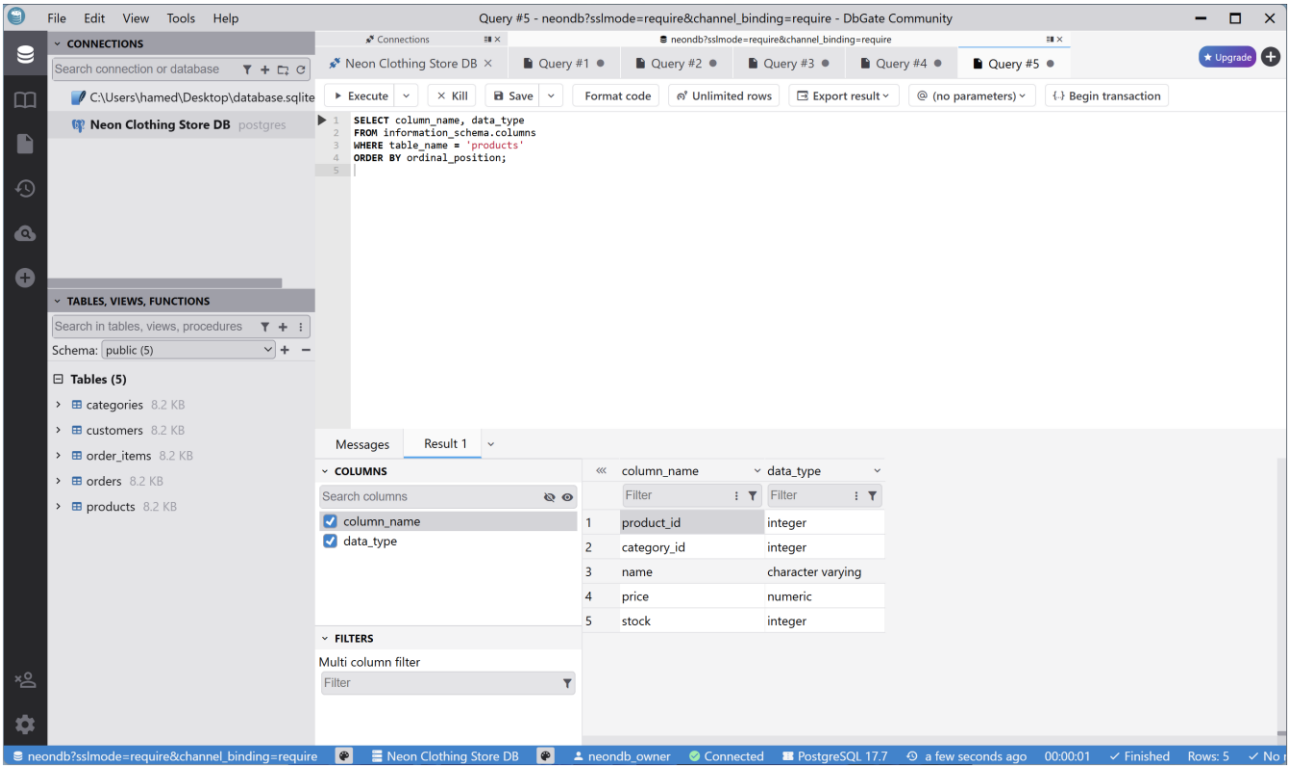
Inside the transaction, the API inserts a new row into the orders table and stores the new order_id. After that, it loops through each item in the order. For every product, it checks that product_id and quantity are valid, then it locks the product row using SELECT ... FOR UPDATE to prevent two orders from reducing the same stock at the same time. If the product does not exist or if there is not enough stock, the API returns an error and the transaction is rolled back.

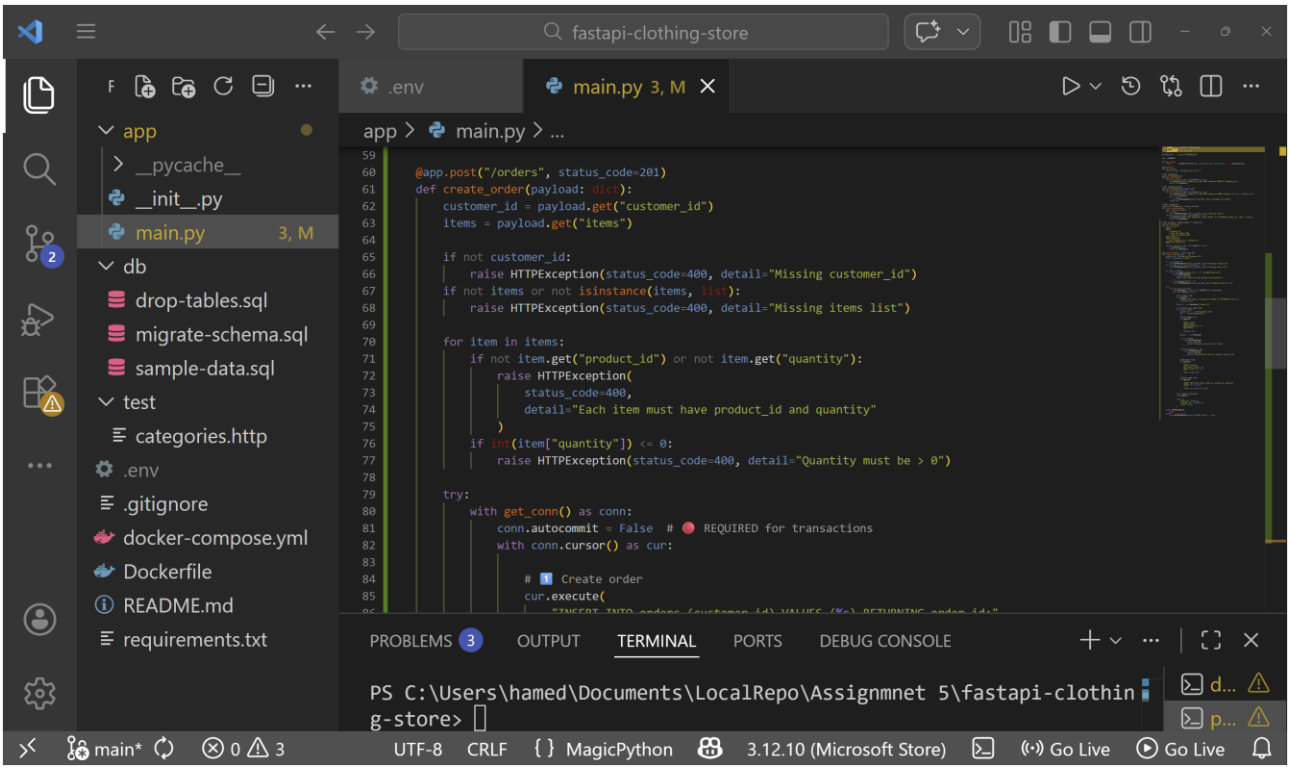
When stock is available, the endpoint updates the products table by subtracting the ordered quantity, and then inserts the item into the order_items table. After all items are processed successfully, the transaction is committed. If any error happens during the process, the transaction is rolled back to avoid partial orders or incorrect stock updates. Although the assignment example shows a single product per request, I implemented support for multiple order items using a list, while preserving the same validation logic.

Finally, I tested the endpoint using Swagger UI with sample orders. Successful requests created an order and reduced product stock correctly, and invalid requests (wrong product ID or insufficient stock) failed without changing the database.

Assignment 5: Clothing Store API with Analytics

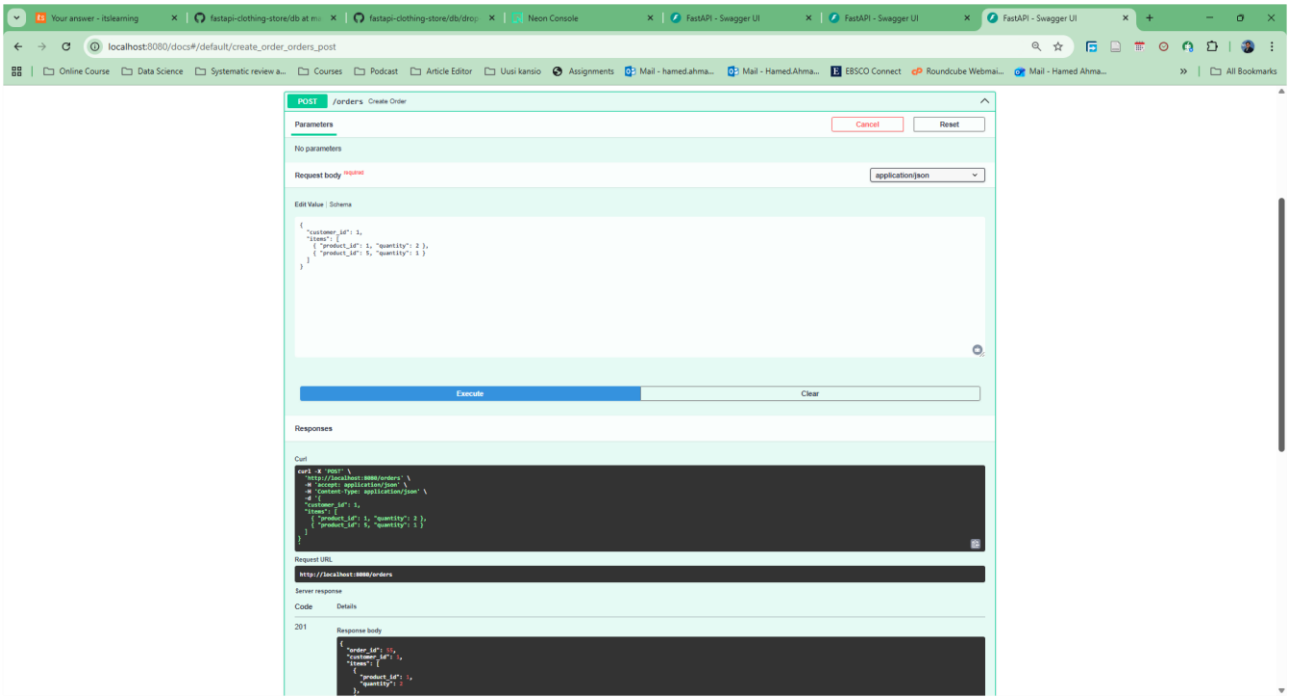
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

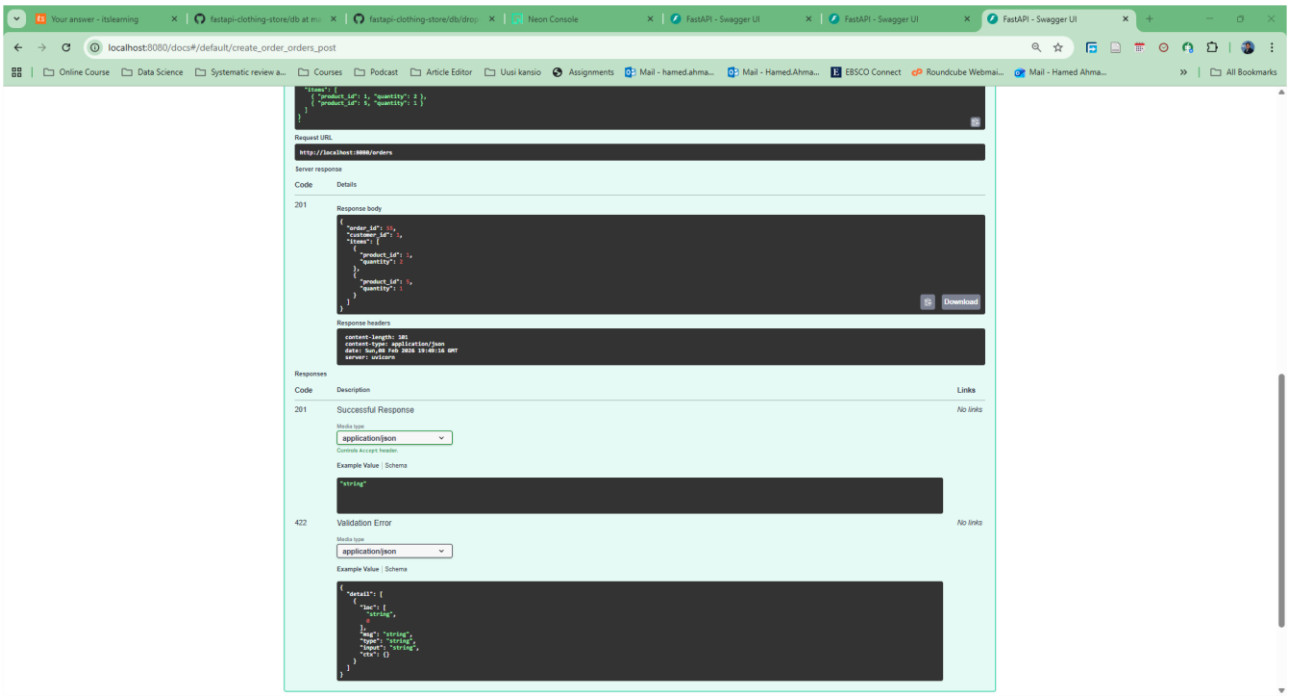




Assignment 5: Clothing Store API with Analytics

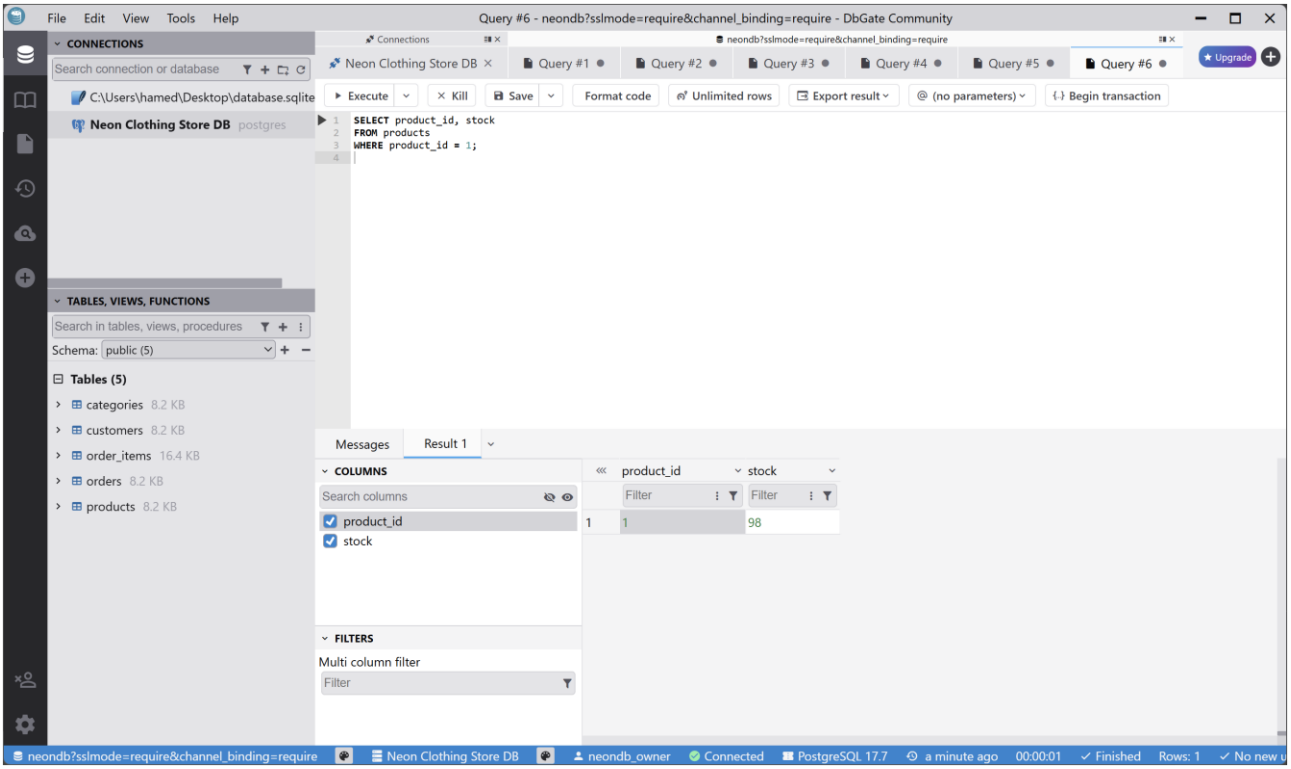
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

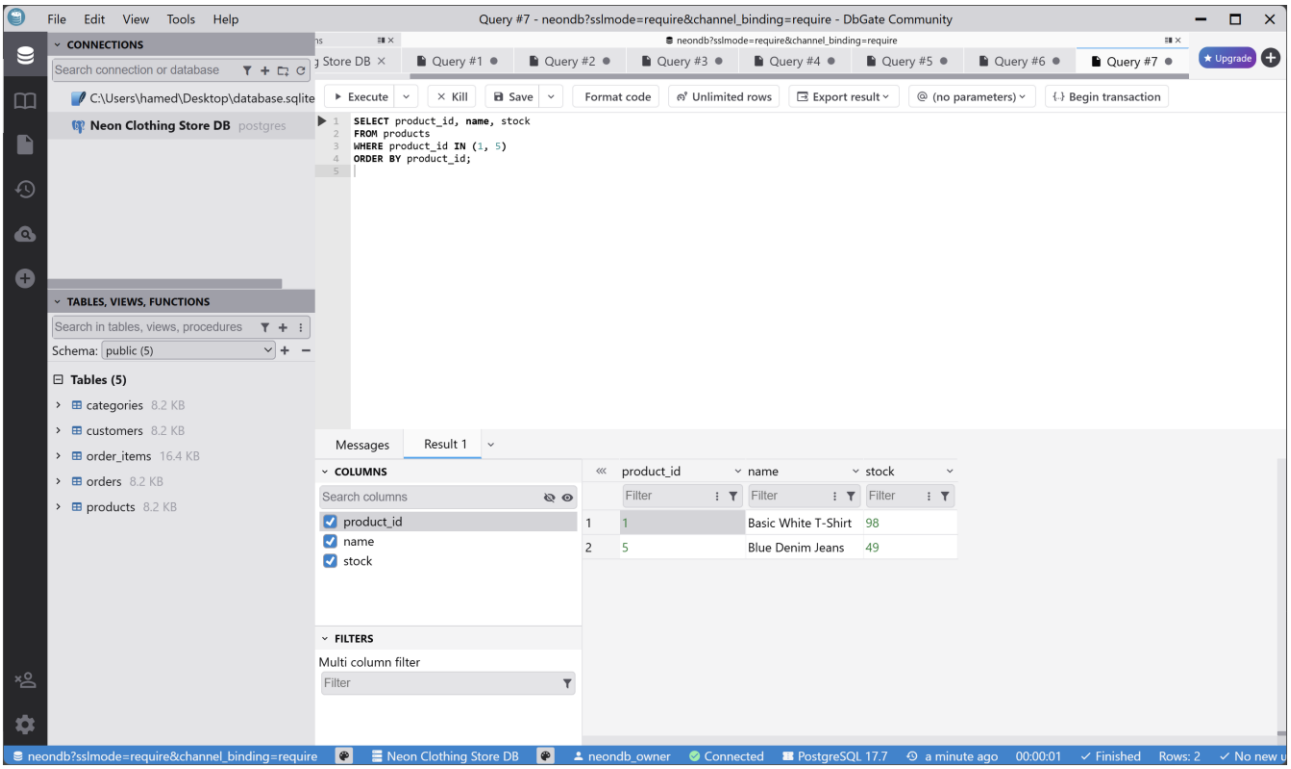




Assignment 5: Clothing Store API with Analytics

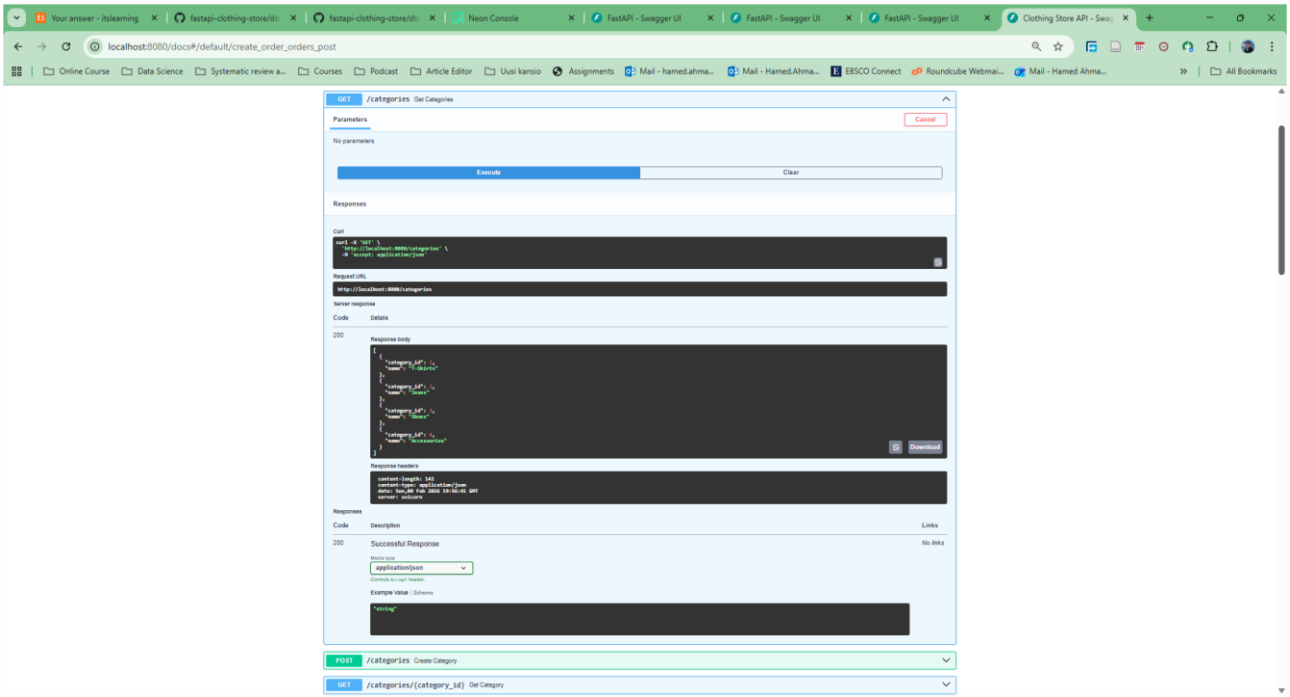
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

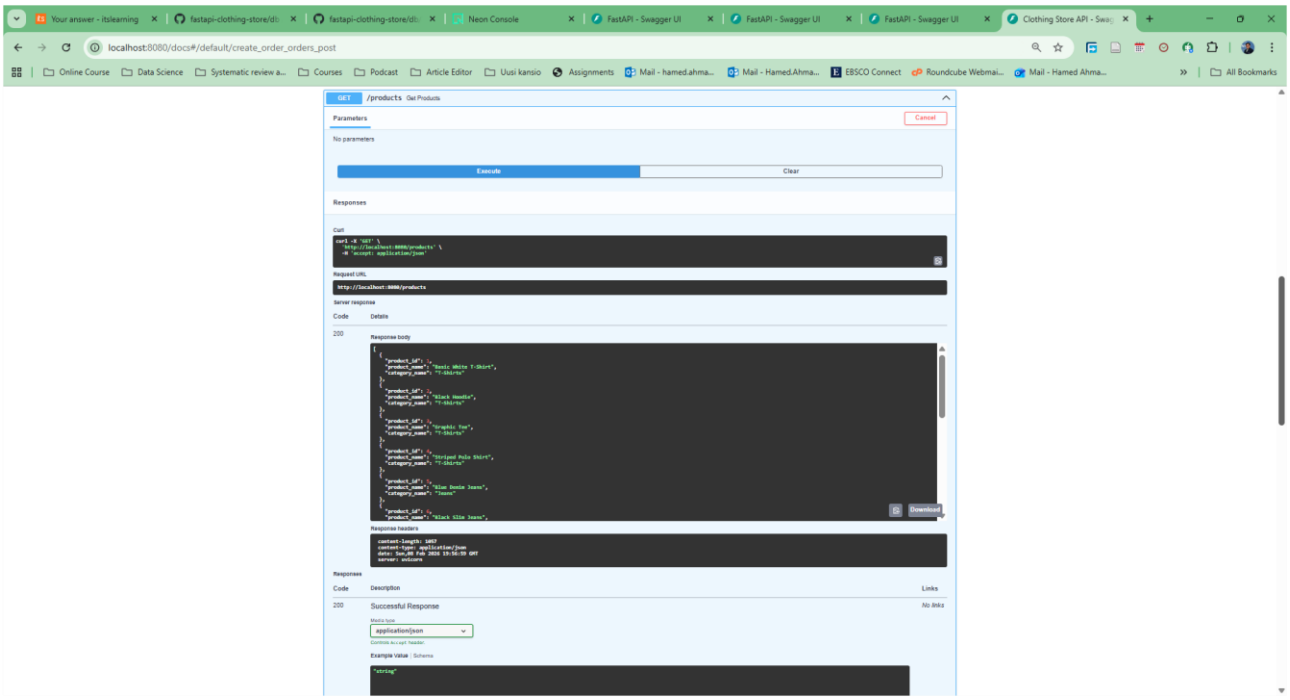


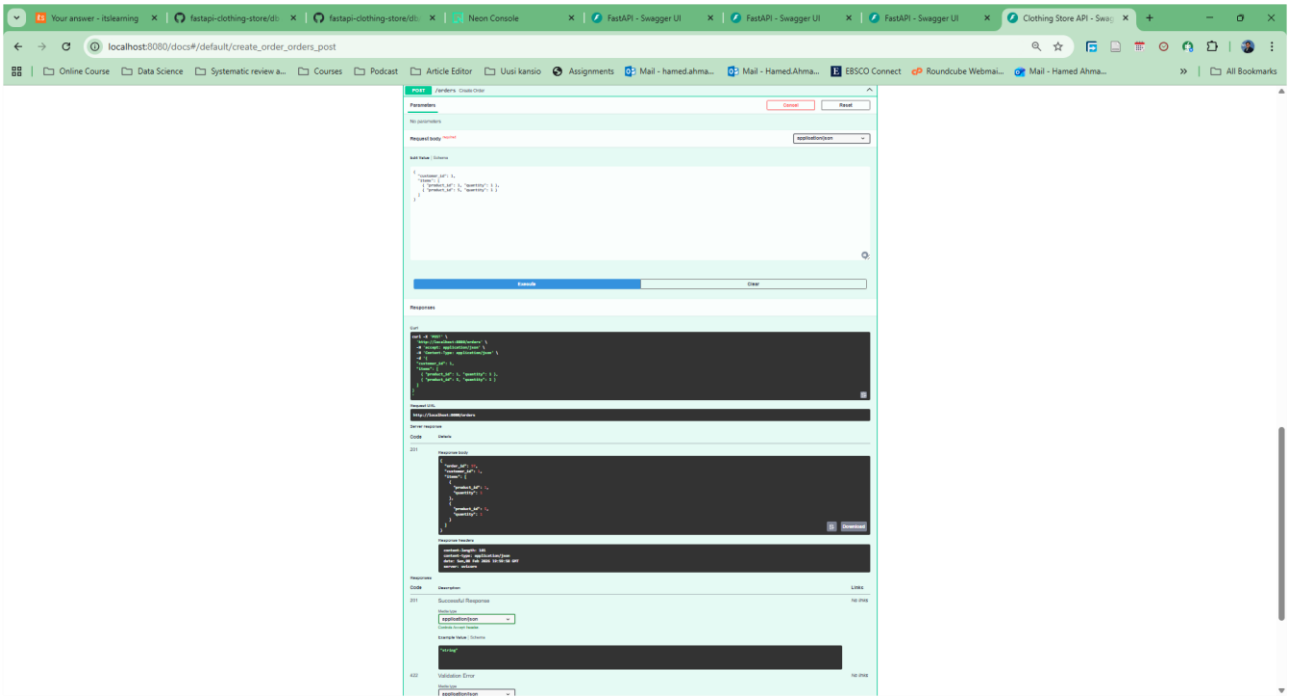


Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia







Step 6 — Analytics endpoints (SQL aggregates + joins)

In this step, I added analytics endpoints that return summary statistics from the database using SQL aggregates and joins. The purpose was to practice JOIN, GROUP BY, and aggregate functions like SUM() and COUNT() to produce useful reports from the order data.

First, I implemented GET /analytics/top-products to show the best-selling products. This endpoint joins order_items with products, groups the results by product, and calculates total_units_sold using SUM(oi.quantity). The results are sorted from highest to lowest and limited to the top five products.

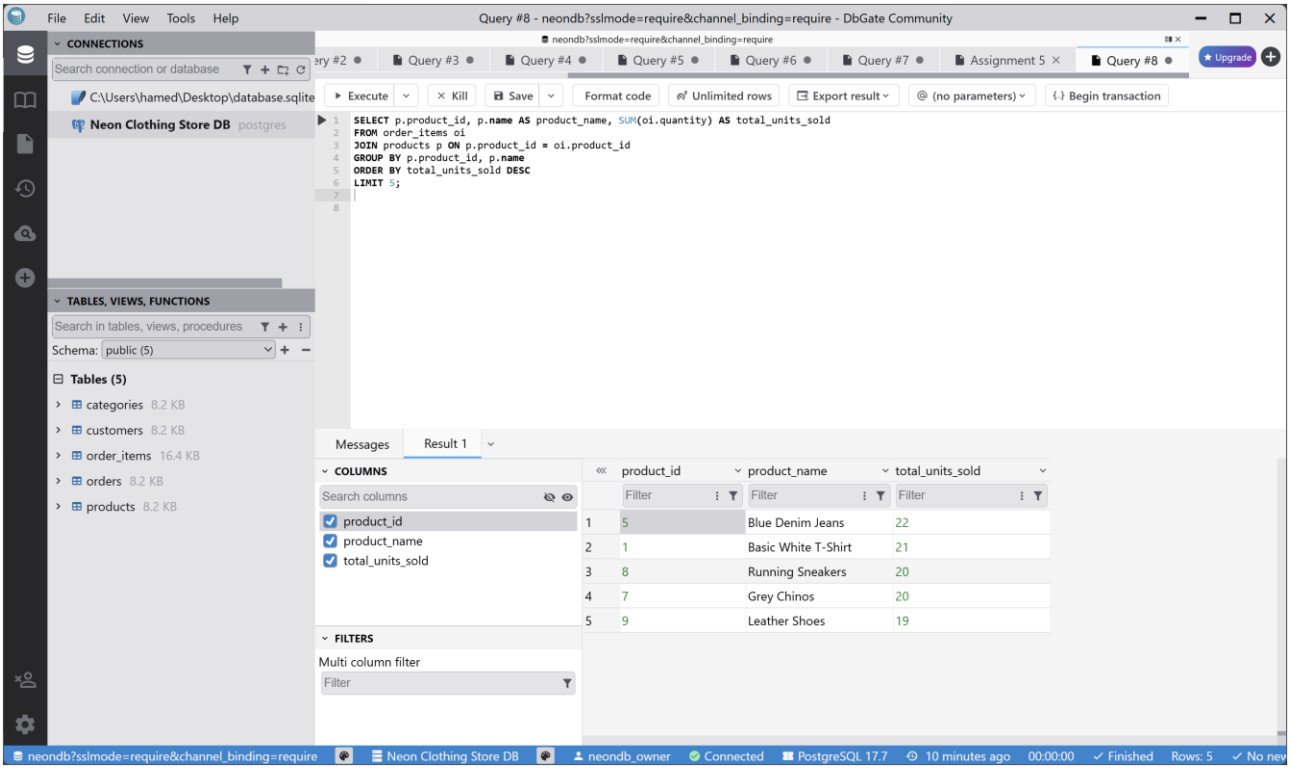
Next, I implemented GET /analytics/category-sales to summarize sales by category. This query joins order_items → products → categories, groups by category, and sums total units sold per category. The output shows which categories have the most sales overall.

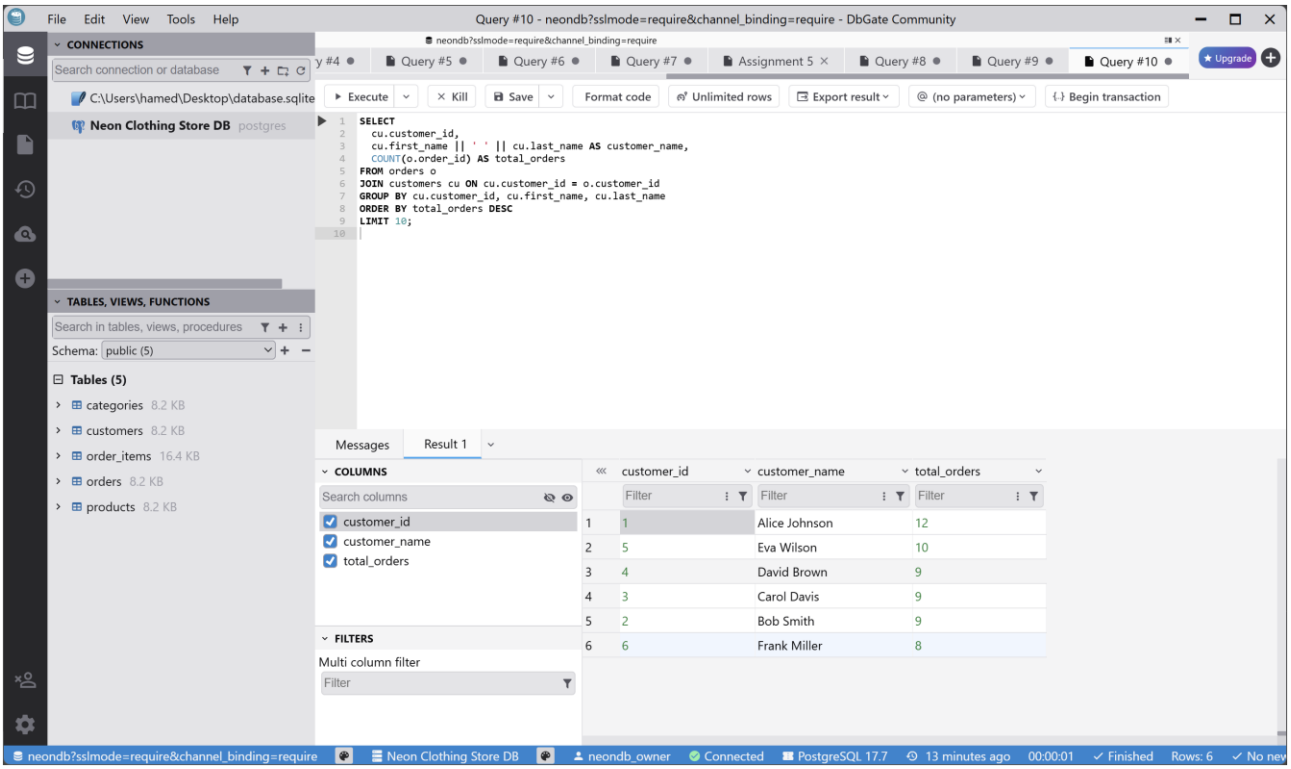
Finally, I implemented GET /analytics/top-customers to find the most active customers. This endpoint joins orders with customers, groups by customer, and counts how many orders each customer has made using COUNT(o.order_id). The results are sorted in descending order and limited to the top ten customers.

All three endpoints reuse the same database connection helper and return the query results as JSON in Swagger UI, which confirmed that the aggregate queries and joins were working correctly. These endpoints fulfill the required statistics functionality, although the route names differ slightly and include additional analytical breakdowns.

Assignment 5: Clothing Store API with Analytics

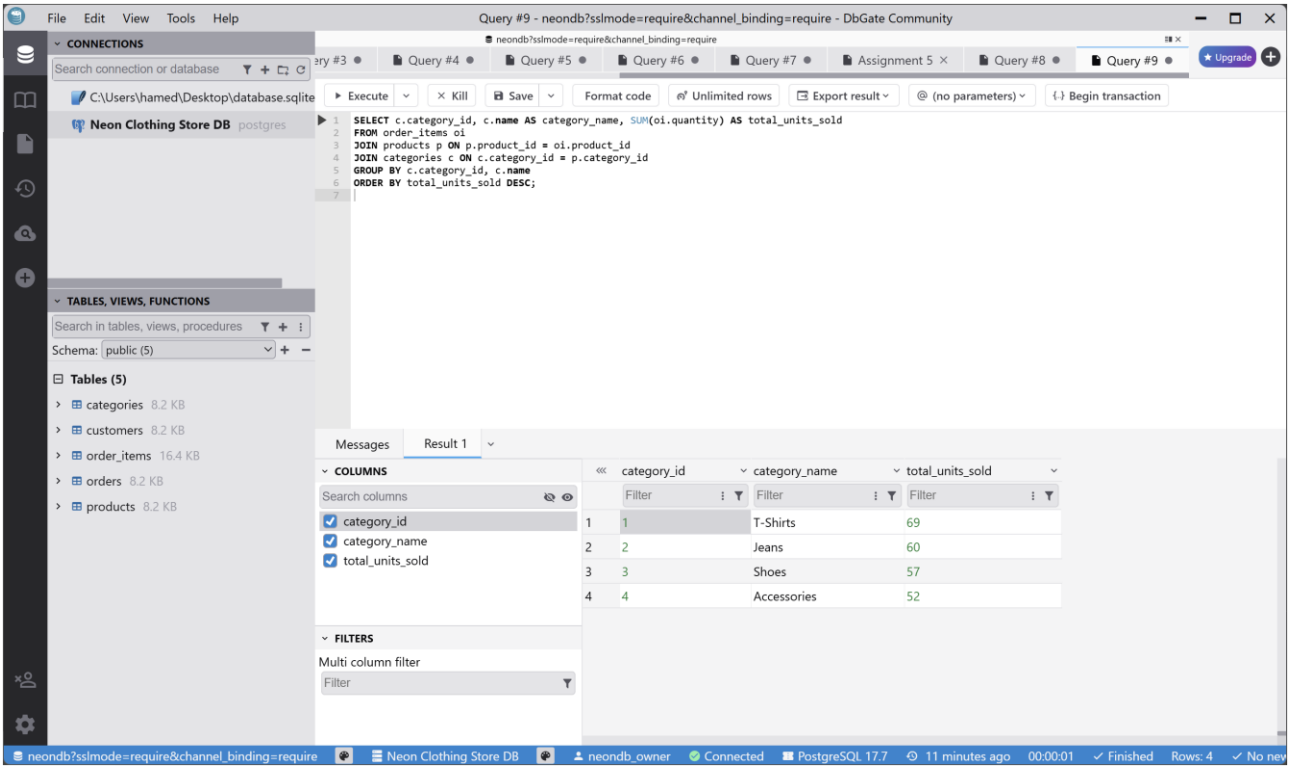
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

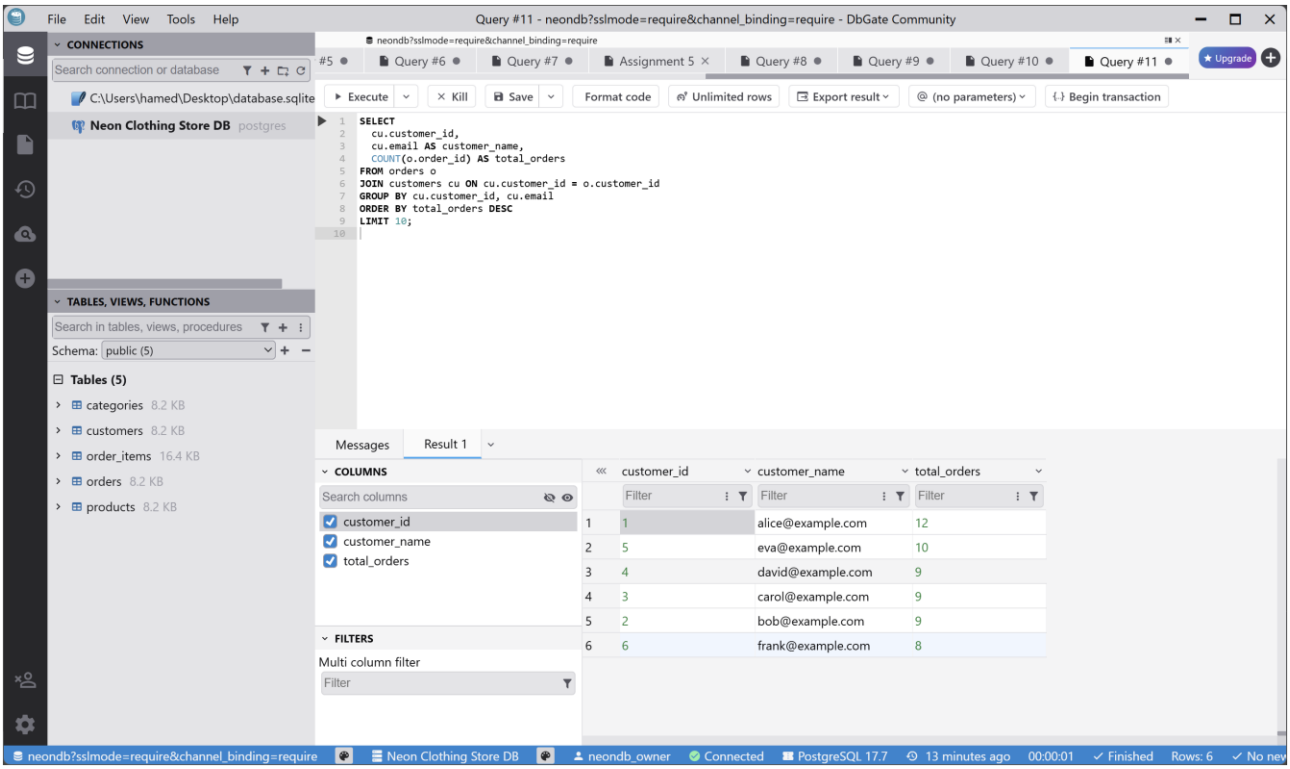




Assignment 5: Clothing Store API with Analytics

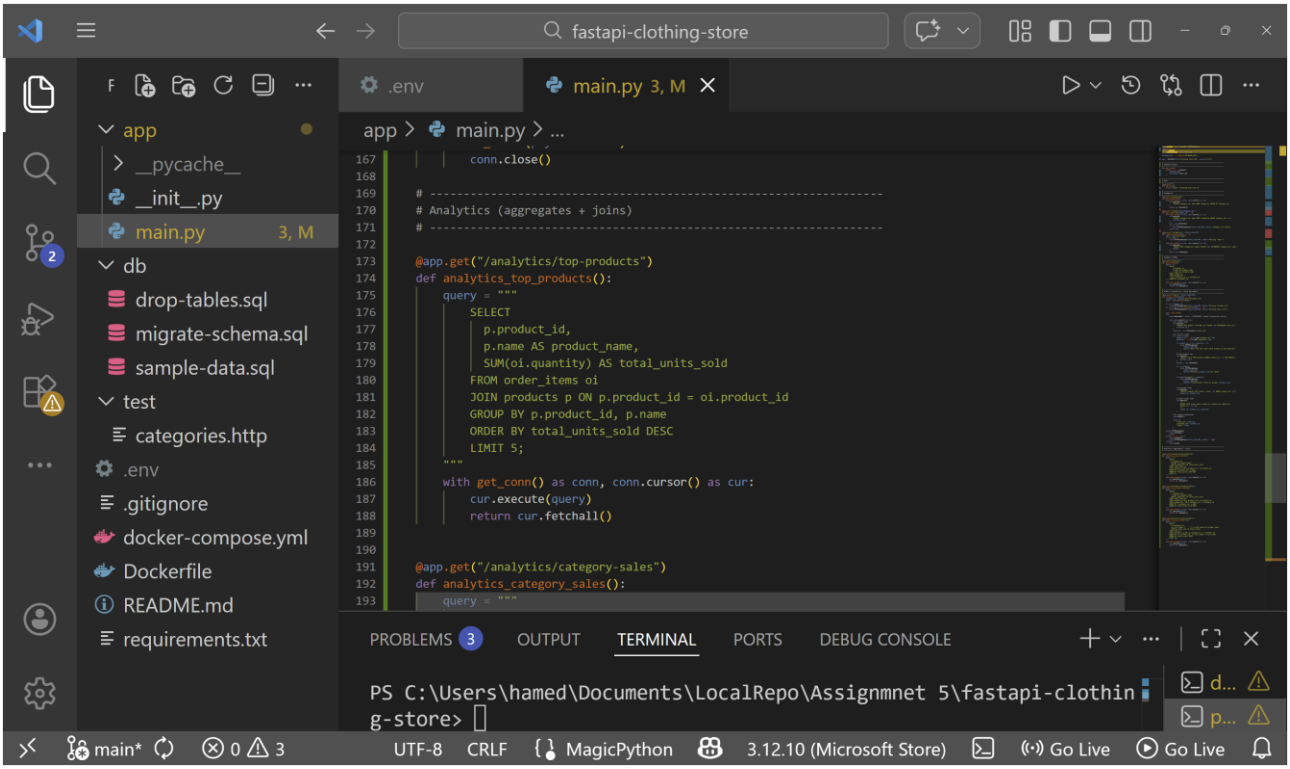
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

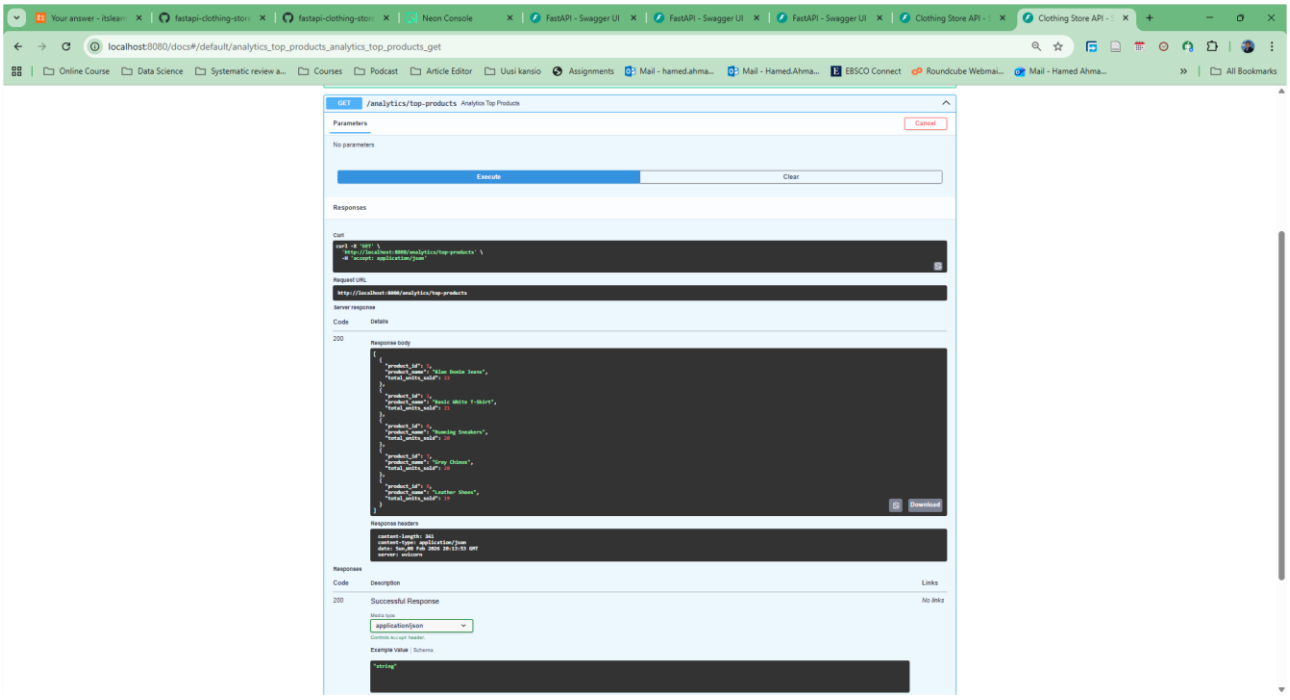




Assignment 5: Clothing Store API with Analytics

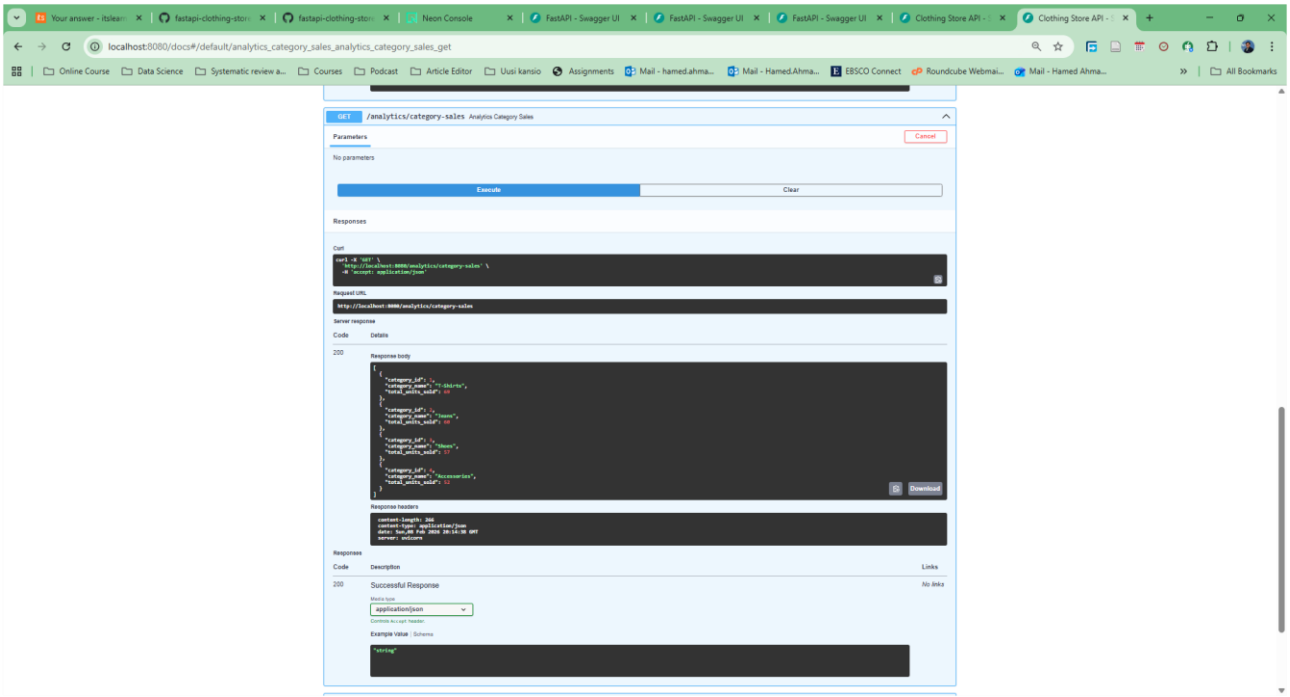
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

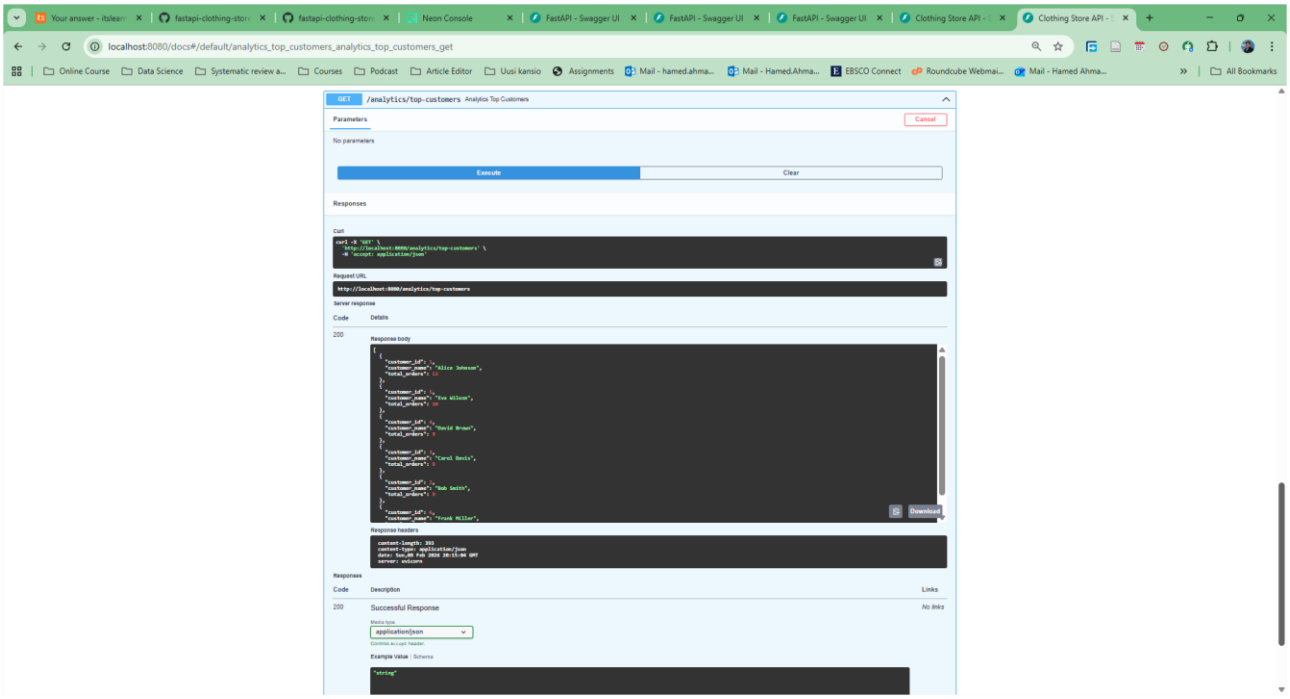




Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





Step 7 (Optional) — Auth (bcrypt + JWT) + RBAC

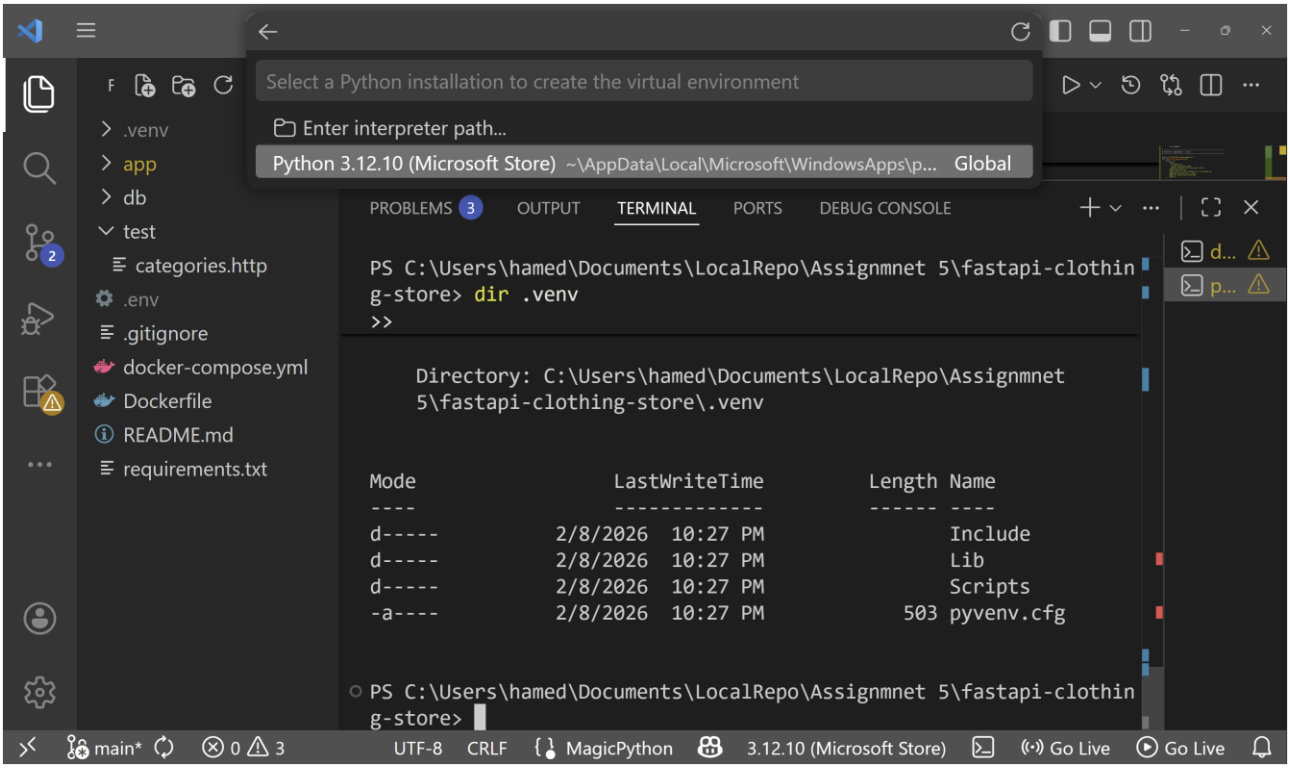
In this step, I added authentication and access control to the Clothing Store API. The goal was to securely register users, allow them to log in, and restrict certain endpoints based on user roles.

I first implemented user registration using the /auth/register endpoint. When a user registers, the password is not stored directly in the database. Instead, it is hashed using bcrypt before being saved. I verified this by checking the customers table in DbGate and confirming that the password was stored as a hash.

Next, I implemented user login with the /auth/login endpoint. Users log in using their email and password. If the credentials are correct, the API returns a JSON Web Token (JWT). This token is later used to access protected endpoints. I tested this in Swagger and confirmed that a token was returned after a successful login.

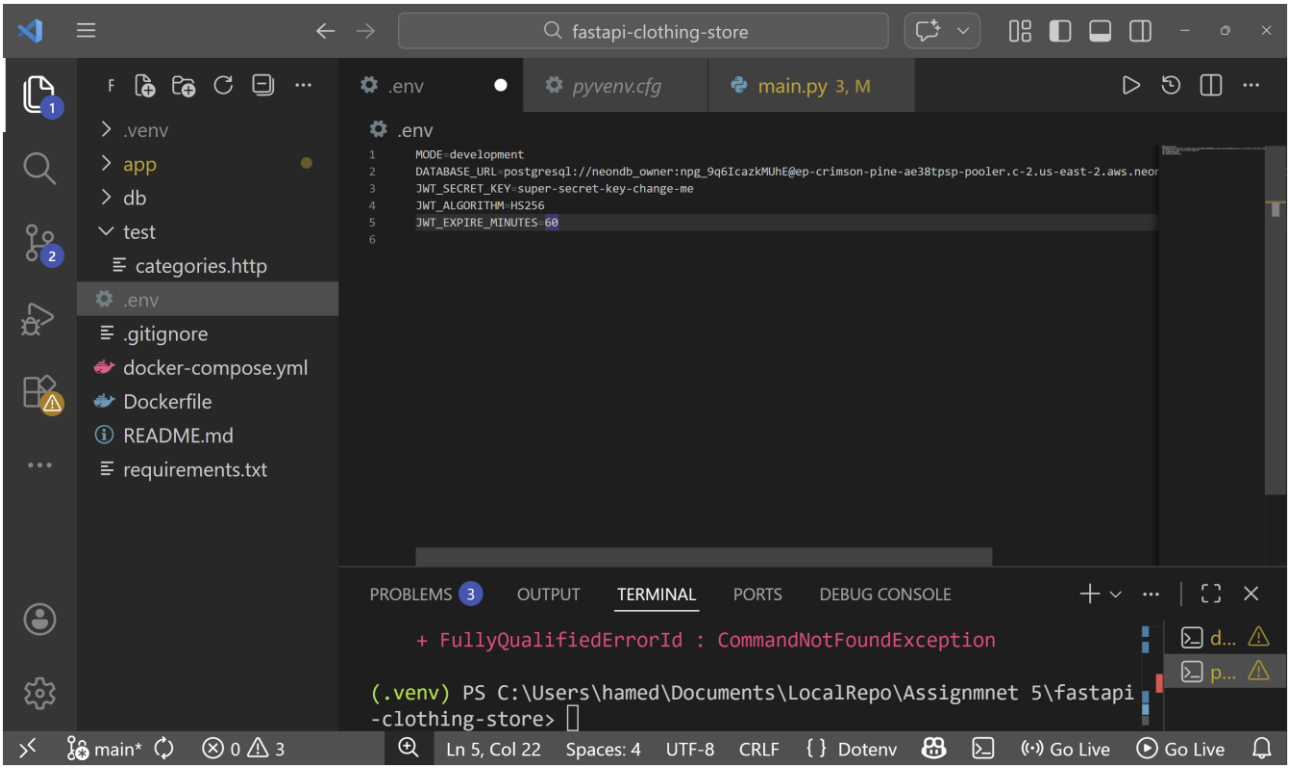
Finally, I added role-based access control. Each user has a role stored in the database (user or admin). Admin-only endpoints, such as statistics and administrative CRUD operations, were protected so that only users with the admin role could access them. I confirmed this by testing the endpoints with and without admin privileges and by checking the user role in DbGate.

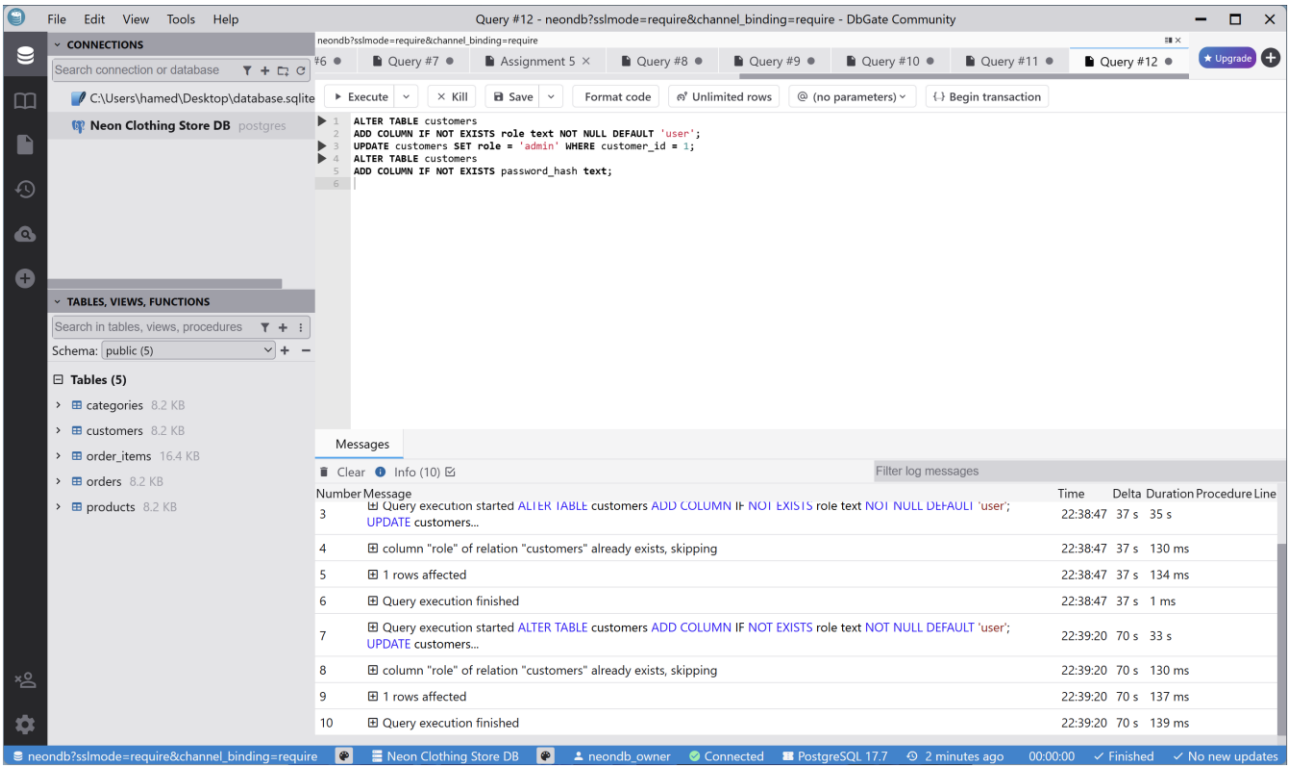
This completed the authentication and authorization setup for the API.



Assignment 5: Clothing Store API with Analytics

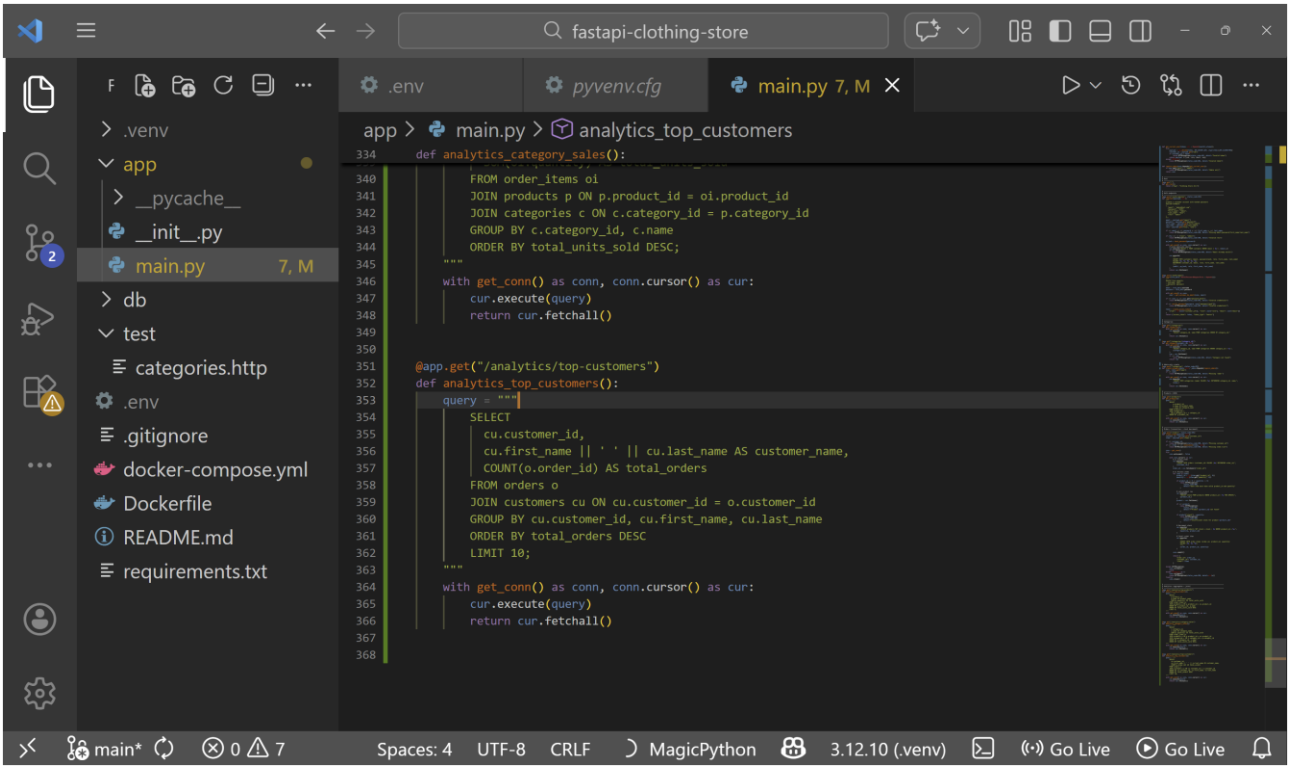
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

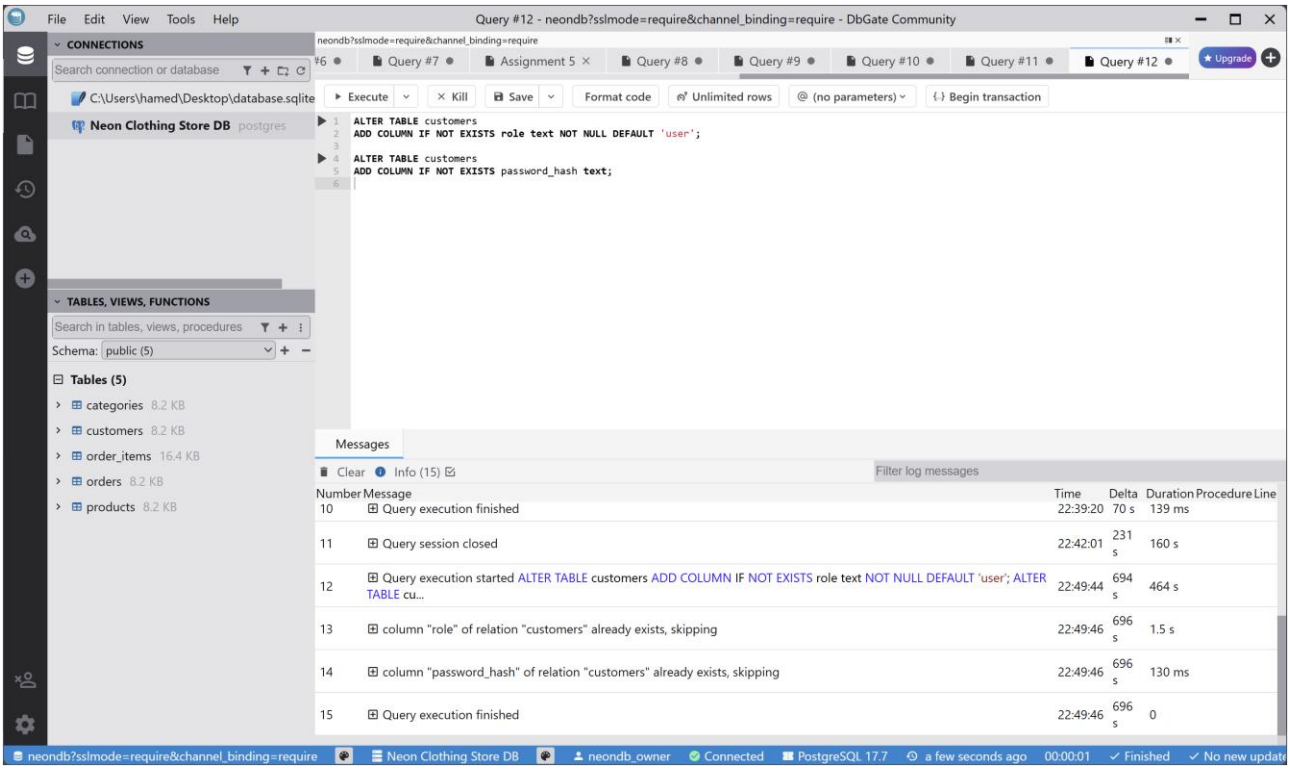




Assignment 5: Clothing Store API with Analytics

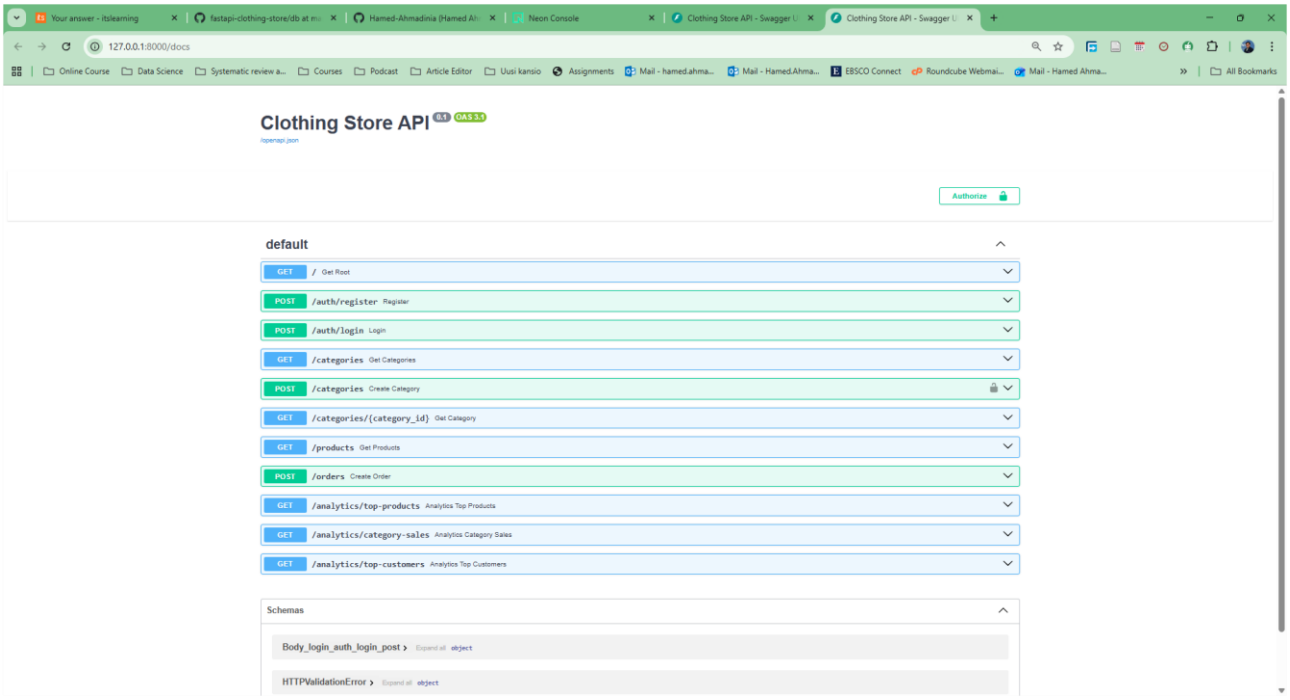
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

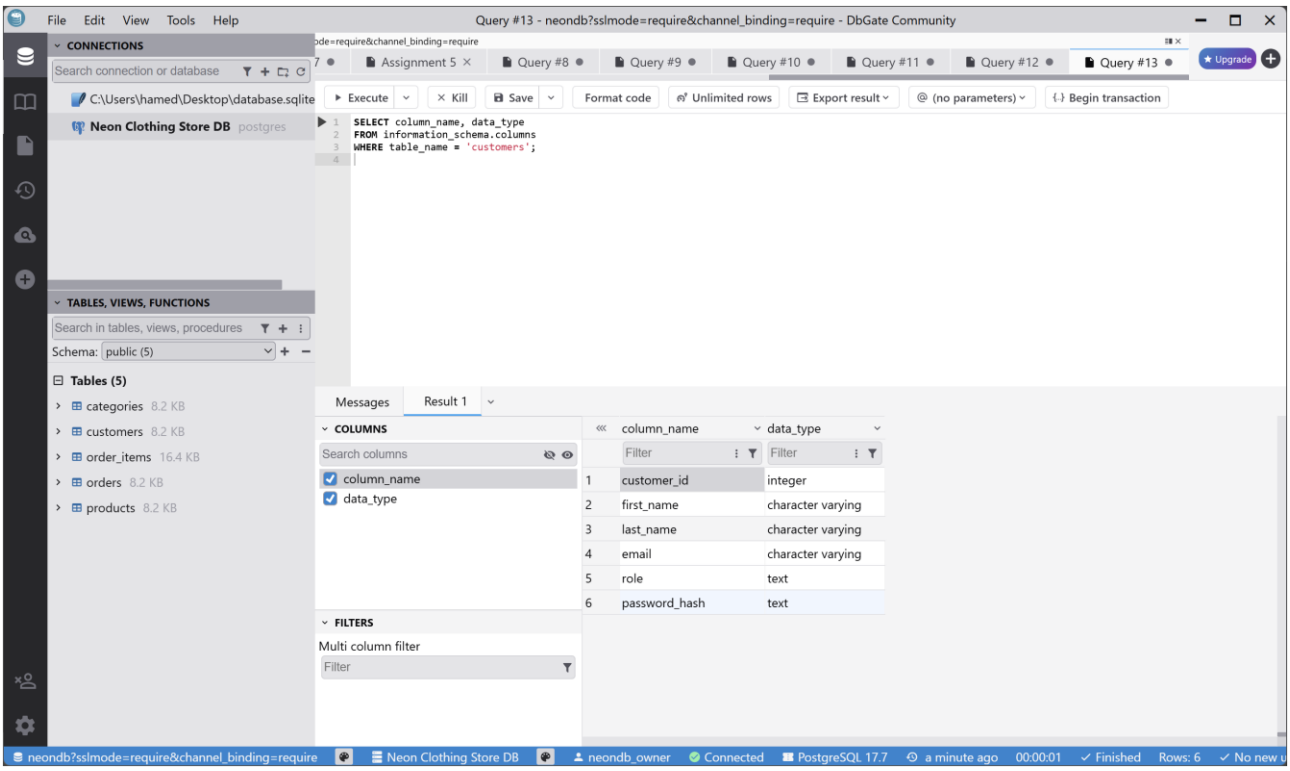




Assignment 5: Clothing Store API with Analytics

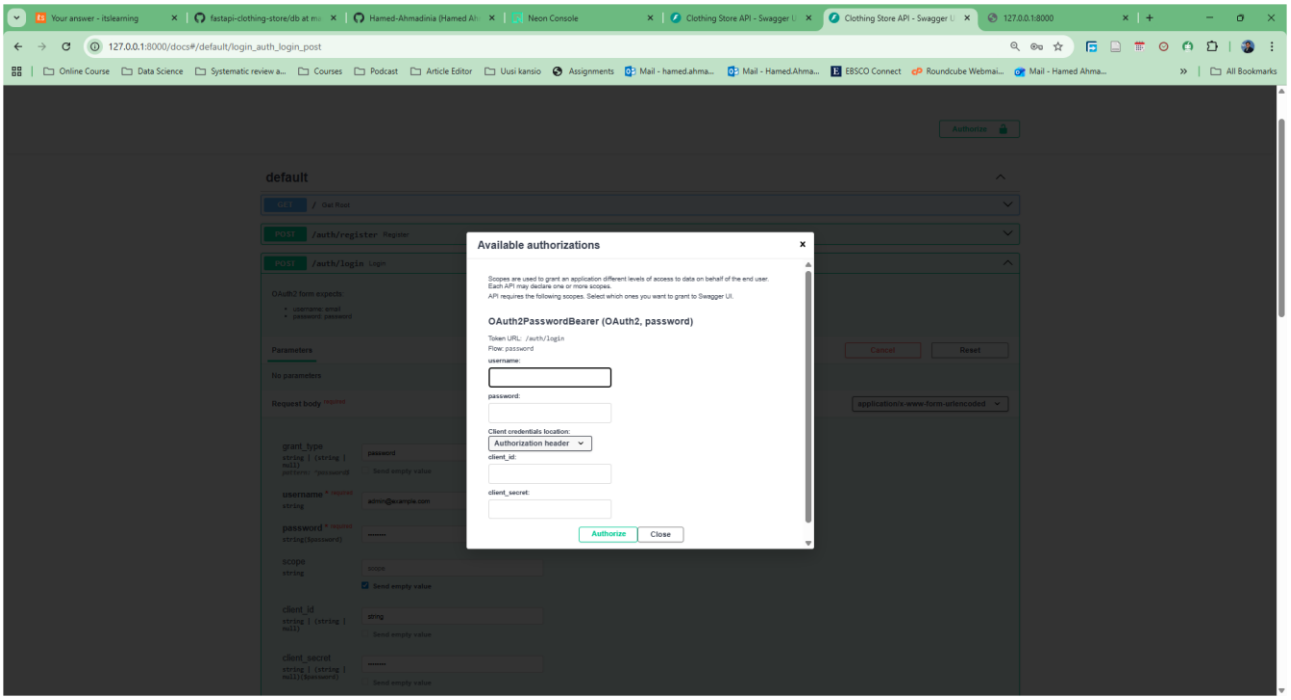
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia

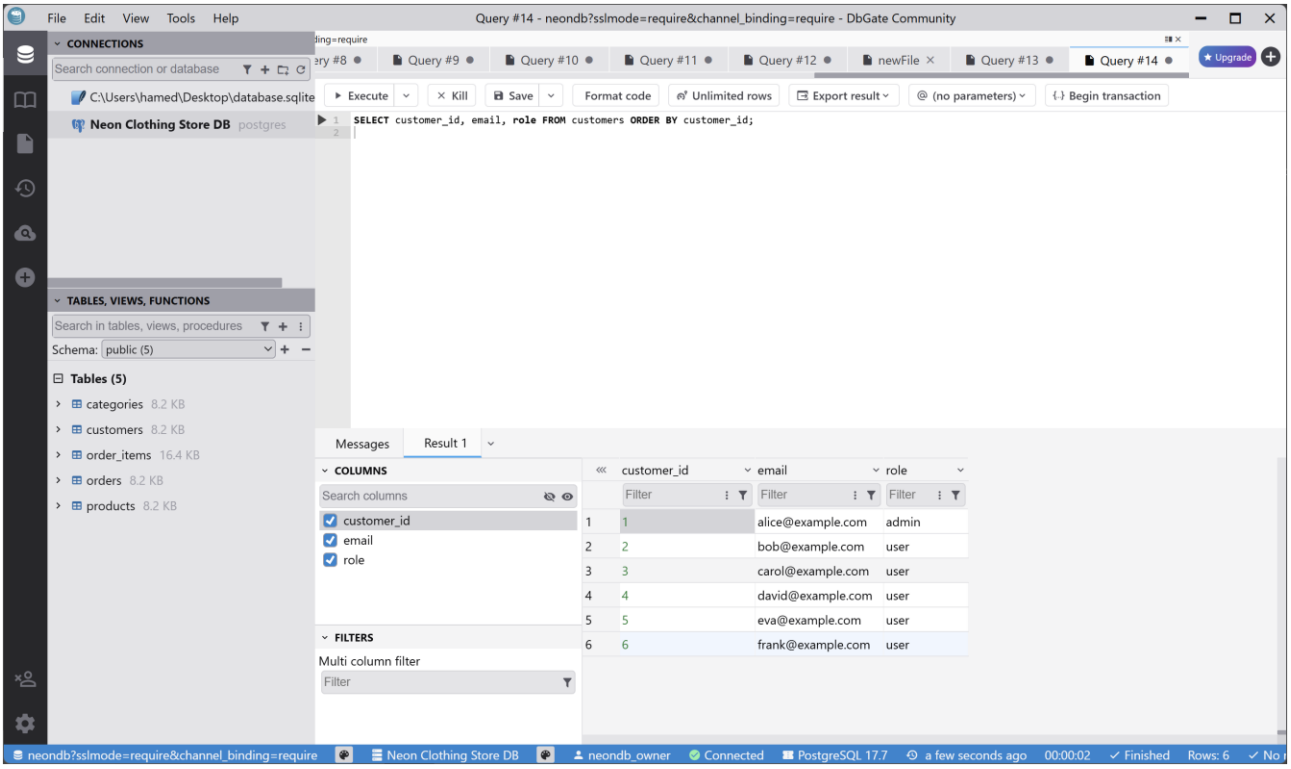




Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia





Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering
By: Hamed Ahmadinia